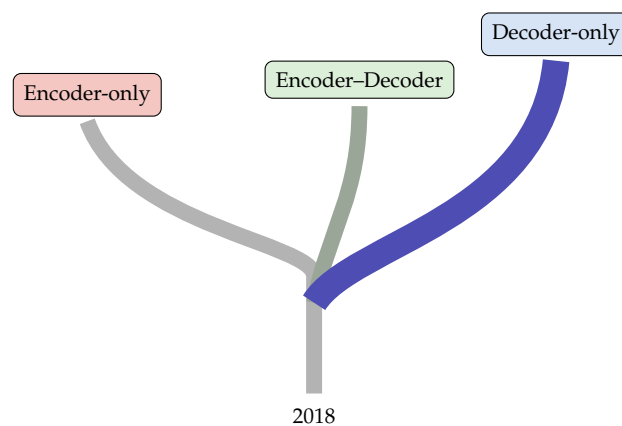


Introduction to Large Language Models

Week 8.1 — Detailed Notes

The Bigger Picture and the Road Ahead (Lecture 6)



Based on the lectures of Prof. Mitesh M. Khapra (AI4Bharat, IIT Madras)

Notes prepared for Deon Dmello — IITM BS, LLM course

Contents

1	The bigger picture: grouping the LLM zoo	4
1.1	Where we are	4
1.2	The LLM evolutionary tree	4
1.2.1	The story the tree tells	5
1.3	Encoder-only models in one paragraph each	6
2	Scaling laws and the race to scale	7
2.1	The question	7
2.2	What is a scaling law?	7
2.3	The compute budget: $C \approx 6ND$	8
2.4	Chinchilla: scale both, equally	8
2.5	The race: models named on the slides	10
3	Decoder-only dominance, alignment and the open/closed split	11
3.1	What the slides say	11
3.2	Why “alignment” was needed	11
3.3	RLHF in InstructGPT	11
3.4	Open vs. closed	12
3.5	So what do all these decoders differ in?	12
4	The design axes of modern decoder-only LLMs	14
4.1	The unifying diagram	14
4.2	Objective: CLM, MLM, Denoising	15
4.3	Optimiser: AdamW, Adafactor, LION	15
4.3.1	Adam and AdamW	15
4.3.2	Adafactor (recap from Week 7)	15
4.3.3	LION (EvoLved Sign Momentum)	15
4.4	Activation functions and the FFN	16
4.4.1	Gated linear units (GLU family)	16
4.5	Dense FFN vs. Mixture of Experts (MoE)	17

4.6	Normalisation: type and placement	18
4.6.1	Types	18
4.6.2	Placement: Post-Norm, Pre-Norm, Sandwich	18
4.7	Attention variants	19
4.7.1	Full vs. sparse attention	19
4.7.2	MHA, MQA, GQA	19
4.7.3	FlashAttention	20
4.7.4	PagedAttention	20
4.8	Position encodings	21
4.8.1	Absolute (sinusoidal or learned)	21
4.8.2	Relative position	21
4.8.3	RoPE: rotary position embedding	21
4.8.4	ALiBi: attention with linear biases	22
4.8.5	NoPE: no position encoding	22
4.9	Depth and training steps	23
4.10	Datasets	23
5	Case studies: GPT-1, GPT-3 175B, PaLM 540B	25
5.1	GPT-1 (2018)	25
5.2	GPT-3 175B (2020)	26
5.3	PaLM 540B (2022)	27
5.4	Side-by-side	28
6	The road ahead	29
6.1	Data sources, data pipelines, datasets	29
6.1.1	What “filtering rules” typically include (preview)	29
A	Revision sheet	31
A.1	Formula sheet	32
A.2	Which model chose what	32
B	Numerical problems with worked solutions	33
C	Subjective questions with model answers	38
D	Multiple-choice questions with answers and explanations	43
D.1	The big picture	43
D.2	Scaling	44
D.3	Design axes	45
D.4	Case studies	47

D.5 Answer key 49

Section 1

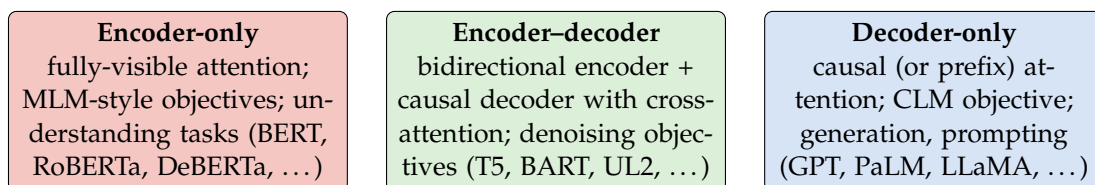
The bigger picture: grouping the LLM zoo

1.1 Where we are

In the previous lecture we took a deep dive into T5 and the design choices behind it. Since then **many LLMs have been released, and the number keeps growing**. This lecture steps back to:

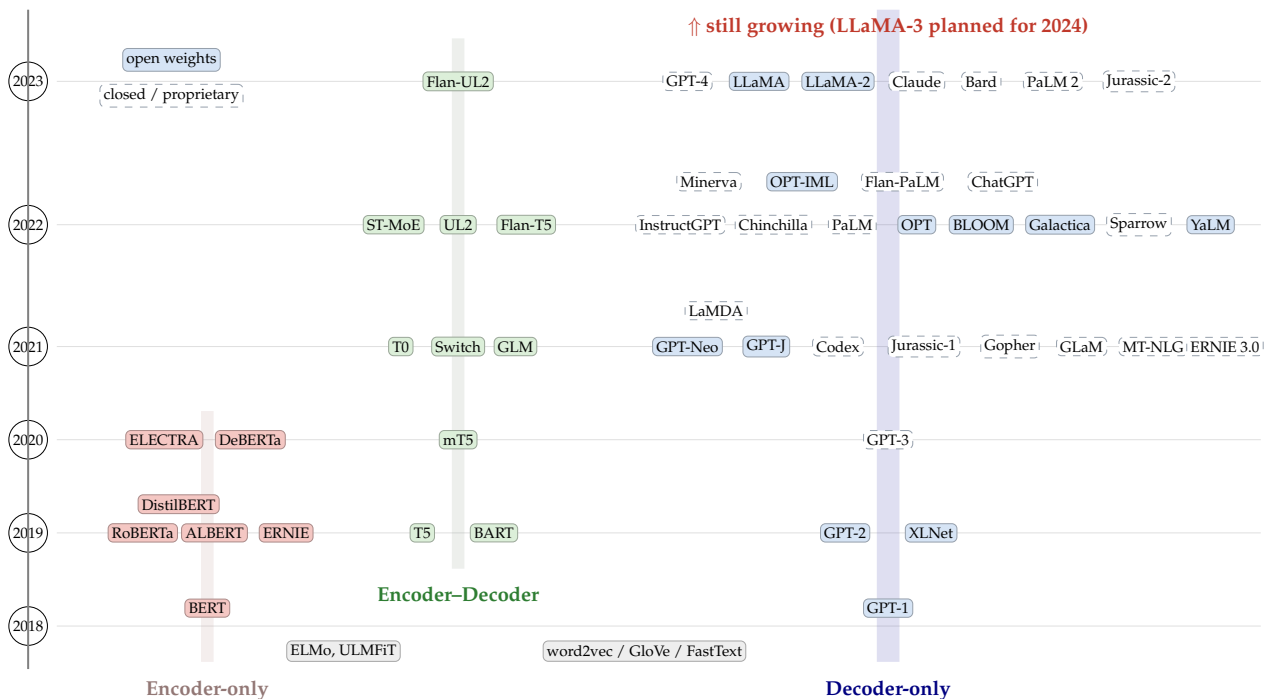
1. **group** the prominent models, and
2. see how they **differ**: which **pre-training datasets** they use and which **components of the architecture** they modify.

One natural grouping is by **architecture type**: **encoder-only**, **encoder-decoder**, or **decoder-only**.



1.2 The LLM evolutionary tree

The lecture builds up the “evolutionary tree” of LLMs (image source: Mooler, i.e. the *LLMs Practical Guide* of Yang et al., 2023) year by year. A simplified redrawing:



Simplified; positions are approximate by year. Filled chip = open-source weights; dashed white chip = closed.

1.2.1 The story the tree tells

1. **2018–2019: encoders dominate.** Encoder-only models (BERT and its variants) **dominated the field for a short time** after BERT's introduction. At the same time, **decoder-only models (GPT)** emerged for language modelling.
2. **It did not last long.** GPT-2 argued for **decoder-only** models (zero-shot prompting) and **T5** for **encoder-decoder** models (text-to-text transfer).
3. **2020: scaling laws.** Two new questions appeared: *should we scale the model, the dataset, or both?* and *in what proportion?* The first answer (Kaplan et al.) favoured model size, which led to a **race to build larger and larger models** (Section 2).
4. **2020–2022: the giants.** GPT-3 (175B), GPT-J (6B), Jurassic-1 (178B), Gopher (280B), GLaM (1.2T, mixture-of-experts).
5. **2022–2023: decoder-only dominance and alignment.** OPT-175B (open), InstructGPT (RLHF), ChatGPT, and then GPT-4, LLaMA, LLaMA-2, Claude, and many more (Section 3).

Intuition : why did the decoder-only branch win?

- **One model for everything:** with prompting, a decoder-only LM can do any task framed as text continuation, so no task-specific heads are needed (GPT-2's insight).
- **Simplest to scale:** a single homogeneous stack, one objective (CLM) that yields a training signal at *every* token, trivially parallel with teacher forcing.
- **Generation is the product:** chat assistants need open-ended generation, which is exactly what a causal LM does natively.
- **Emergent in-context learning** appeared at scale in GPT-3 and reinforced the trend.

Recall from Week 7 that at *equal compute and small scale* T5 found enc–dec slightly better. The field’s preference for decoder-only is about *simplicity, generality and scaling*, not a contradiction of that result.

Note on the slides

The slide says “LLaMA-3 is planned for 2024!” The lecture predates its release; Meta released LLaMA-3 in April 2024. Treat the tree as a snapshot of the field in late 2023.

1.3 Encoder-only models in one paragraph each

Since the tree names them, here is what each encoder-only variant changed relative to BERT (beyond the slides, for context):

Model	Key idea
RoBERTa	Same architecture, better training: more data (160GB), longer training, bigger batches, dynamic masking, no NSP.
ALBERT	Parameter efficiency: factorised embeddings ($V \times E$, $E \ll d_{\text{model}}$) and cross-layer parameter sharing; sentence-order prediction.
DistilBERT	Knowledge distillation: 6 layers, 40% smaller, about 97% of BERT’s GLUE performance.
ERNIE (Baidu)	Knowledge masking: mask whole entities and phrases instead of random tokens.
ELECTRA	Replaced-token detection: a small generator corrupts tokens and the main model classifies <i>every</i> token as original or replaced, so there is signal at all positions, not just 15%.
DeBERTa	Disentangled attention (content and relative position handled separately) and an enhanced mask decoder.

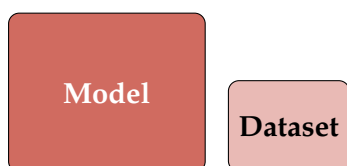
Section 2

Scaling laws and the race to scale

2.1 The question

After GPT-2 and T5 showed that bigger models trained on more data do better, two questions arose (slide):

- Should we **scale the model, the dataset, or both**?
- What is the **proportion** of scaling?



The slide draws the *model* box much larger than the *dataset* box: the first “scaling laws” (Kaplan et al., 2020) said that, for a fixed compute budget, most extra compute should go into **model size**. This triggered the race to build ever larger models.

2.2 What is a scaling law?

A scaling law is an empirical **power-law** relation between test loss L and a resource: parameters N , dataset tokens D , or compute C . It holds over many orders of magnitude.

Beyond the slides : Kaplan et al. (2020) scaling laws

When one resource is the bottleneck (the others are ample),

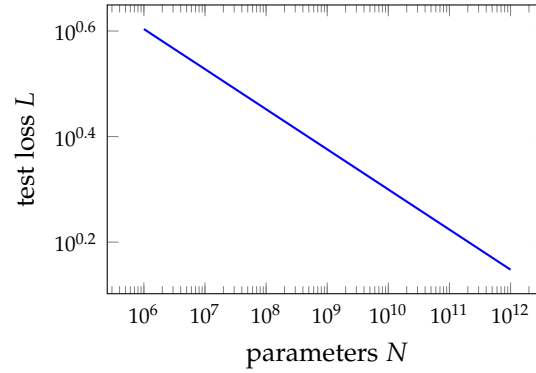
$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \alpha_N \approx 0.076; \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \alpha_D \approx 0.095; \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}, \alpha_C \approx 0.050.$$

Here N excludes embeddings, and N_c , D_c , C_c are fitted constants. For a fixed compute budget they found the optimal allocation

$$N_{\text{opt}} \propto C^{0.73}, \quad D_{\text{opt}} \propto C^{0.27}.$$

So $10\times$ more compute should buy about $5.4\times$ more parameters but only about $1.9\times$ more data. That is the “Model $\gg$ Dataset” picture on the slide.

Power law = straight line on log-log axes



Derivation : why a power law is a straight line on log-log axes

$L = (N_c/N)^\alpha \Rightarrow \log L = \alpha \log N_c - \alpha \log N$. This is linear in $\log N$ with slope $-\alpha$. Doubling N multiplies the loss by $2^{-\alpha}$; with $\alpha = 0.076$ that is ≈ 0.949 , a 5% reduction per doubling, and it keeps going.

2.3 The compute budget: $C \approx 6ND$

Derivation : FLOPs per token

For a dense transformer with N parameters, a forward pass costs $\approx 2N$ FLOPs per token (a multiply and an add per weight). The backward pass costs about twice the forward ($\approx 4N$): gradients with respect to activations and with respect to weights. Total $\approx 6N$ per token, so for D training tokens

$$C \approx 6ND.$$

GPT-3: $6 \times 175 \times 10^9 \times 300 \times 10^9 \approx 3.15 \times 10^{23}$ FLOPs.

2.4 Chinchilla: scale both, equally

Beyond the slides : Hoffmann et al. (2022), "Chinchilla"

Fit the loss as a function of *both* N and D :

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad E = 1.69, A = 406.4, B = 410.7, \alpha = 0.34, \beta = 0.28.$$

E is the irreducible entropy of text; the other two terms are the penalties for a finite model and finite data.

Derivation : compute-optimal allocation

Minimise $L(N, D)$ subject to $6ND = C$. Substitute $D = C/(6N)$:

$$L(N) = E + AN^{-\alpha} + B\left(\frac{C}{6N}\right)^{-\beta} = E + AN^{-\alpha} + B6^\beta C^{-\beta} N^\beta.$$

Set $\frac{dL}{dN} = 0$:

$$-\alpha AN^{-\alpha-1} + \beta B 6^\beta C^{-\beta} N^{\beta-1} = 0 \Rightarrow N^{\alpha+\beta} = \frac{\alpha A}{\beta B} \left(\frac{C}{6}\right)^\beta.$$

Hence

$$N_{\text{opt}} \propto C^{\frac{\beta}{\alpha+\beta}}, \quad D_{\text{opt}} = \frac{C}{6N_{\text{opt}}} \propto C^{\frac{\alpha}{\alpha+\beta}}.$$

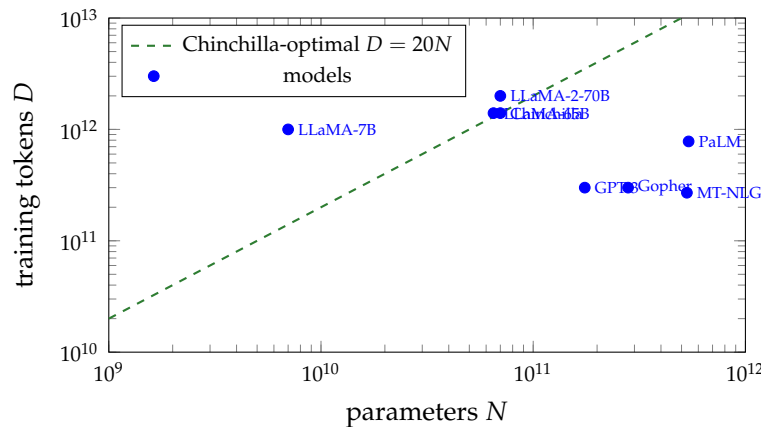
With $\alpha = 0.34, \beta = 0.28$: exponents 0.45 and 0.55, close to 0.5 each. The paper's other two estimation methods gave almost exactly 0.5/0.5. **Parameters and tokens should grow in equal proportion**, at roughly

$$D_{\text{opt}} \approx 20 N_{\text{opt}} \quad (\text{about 20 tokens per parameter}).$$

Derivation : sizing a compute-optimal model

With $D = 20N$: $C = 6N(20N) = 120N^2 \Rightarrow N = \sqrt{C/120}$.

- Gopher's budget $C \approx 5.76 \times 10^{23}$: $N = \sqrt{4.8 \times 10^{21}} \approx 6.9 \times 10^{10} \approx \mathbf{70B}$, $D \approx 1.4T$ tokens. This is exactly Chinchilla (70B, 1.4T), which beat the $4\times$ larger Gopher (280B, 300B tokens).
- GPT-3's budget 3.15×10^{23} : $N \approx 51B$, $D \approx 1.0T$. GPT-3 used 175B parameters and only 300B tokens (≈ 1.7 tokens/parameter), so it was heavily *under-trained* by this criterion.



Models far below the dashed line are “over-sized, under-trained”. After Chinchilla, models such as LLaMA deliberately trained smaller models on many more tokens (even *beyond* 20 tokens/param) because a smaller model is cheaper at inference.

2.5 The race: models named on the slides

Model (year)	Size	Notes
GPT-3 (2020)	175B	Proprietary (OpenAI). 96 layers; showed few-shot in-context learning.
GPT-J (2021)	6B	Open-sourced (EleutherAI), trained on The Pile; uses RoPE and parallel attention/FFN.
Jurassic-1 (2021)	178B	Proprietary (AI21 Labs); large 256k-token vocabulary.
Gopher (2021)	280B	Proprietary (DeepMind); MassiveText dataset, RM-SNorm, relative positions.
GLaM (2021)	1200B	Proprietary (Google); mixture of experts , 64 experts, top-2, about 97B parameters active per token.
OPT (2022)	175B	Open-sourced by Meta; a GPT-3-scale replica with a public logbook.

Watch out : “1200B parameters” is not “1200B of compute”

GLaM is a sparse MoE: each token is routed to 2 of 64 experts, so only $\approx 97\text{B}$ of its 1.2T parameters ($\approx 8\%$) are used per token. Its per-token FLOPs are closer to a 100B dense model. When comparing sizes, always ask whether the model is *dense* or *sparse*.

Key takeaway

Scaling laws made LLM progress *predictable*: loss falls as a power law in N , D and C . Kaplan’s version favoured model size and triggered the size race (GPT-3, Jurassic-1, Gopher, GLaM). Chinchilla corrected the proportion: scale parameters and tokens *together* (≈ 20 tokens per parameter).

Section 3

Decoder-only dominance, alignment and the open/closed split

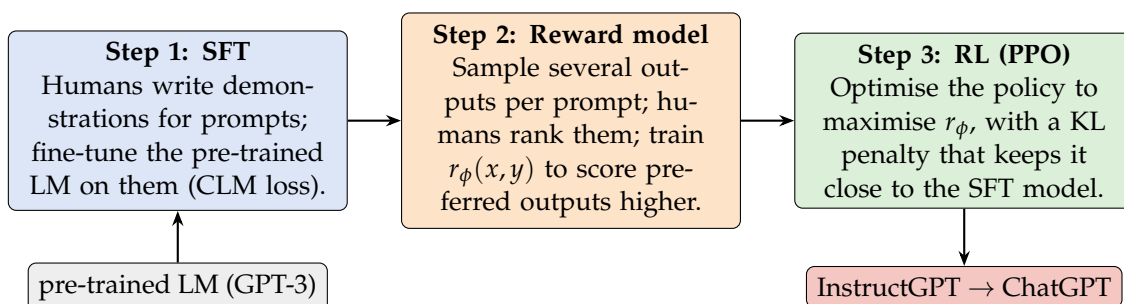
3.1 What the slides say

- Decoder-only models dominated the field.
- Meta released OPT with 175B parameters, **open-sourced**.
- InstructGPT proposed an **RLHF-based alignment objective** for conversational models.
- ChatGPT was built on top of the ideas in InstructGPT.
- The number of **open and closed** LLMs is still growing (LLaMA-3 planned for 2024).

3.2 Why “alignment” was needed

A pre-trained LM is trained to *continue text the way the internet would*, not to *follow instructions helpfully*. Asked “Explain RoPE to a student”, it may continue with more questions, a forum thread, or something unsafe, all of which are plausible continuations of web text. **Alignment** closes the gap between “likely text” and “text a user actually wants”.

3.3 RLHF in InstructGPT



Beyond the slides : the three losses

SFT: $\mathcal{L}_{\text{SFT}} = -\sum_t \log \pi_\theta(y_t \mid x, y_{<t})$ on human demonstrations.

Reward model (Bradley–Terry pairwise loss, where y_w is preferred over y_l):

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right].$$

RL objective, optimised with PPO:

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{SFT}}(\cdot | x)).$$

The KL term stops the policy from drifting into gibberish that “hacks” the reward model.

Intuition : why rankings rather than scores?

Humans are inconsistent at giving absolute scores (“is this a 6 or a 7?”) but fairly consistent at saying which of two answers is better. The Bradley–Terry model converts pairwise preferences into a scalar reward: $P(y_w \succ y_l) = \sigma(r_w - r_l)$. A striking InstructGPT result: labellers preferred the 1.3B aligned model to the 175B GPT-3, so alignment can be worth more than $100\times$ in parameters for user-facing quality.

3.4 Open vs. closed

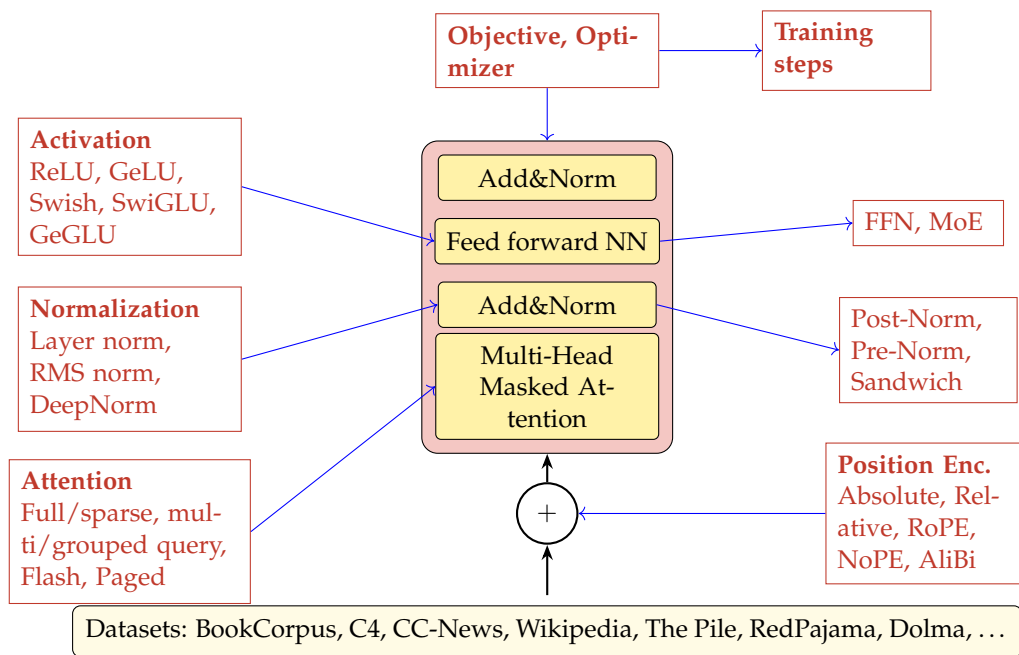
Closed / proprietary	Open weights
GPT-3, Jurassic-1, Gopher, GLaM, PaLM, Chinchilla, ChatGPT, GPT-4, Claude, Bard	GPT-J, GPT-NeoX, OPT, BLOOM, Galactica, LLaMA, LLaMA-2, Falcon, Mistral
Accessible only via API; architecture and data details often undisclosed	Weights downloadable; can be fine-tuned, quantised and studied

3.5 So what do all these decoders differ in?

Even if we keep only the **decoder-only branch** of the tree, there are **dozens of models**. They share the same macro-architecture (a stack of masked-attention + FFN blocks), so what distinguishes them? According to the lecture:

1. **The dataset** used to train the model. **Common Crawl** is the source for the majority of datasets.
2. **Various design choices to reduce the computational budget and increase performance**: activation, normalisation (type and placement), attention variant, position encoding, FFN vs. MoE, objective, optimiser, number of layers and training steps.

The next section goes through every one of these axes.

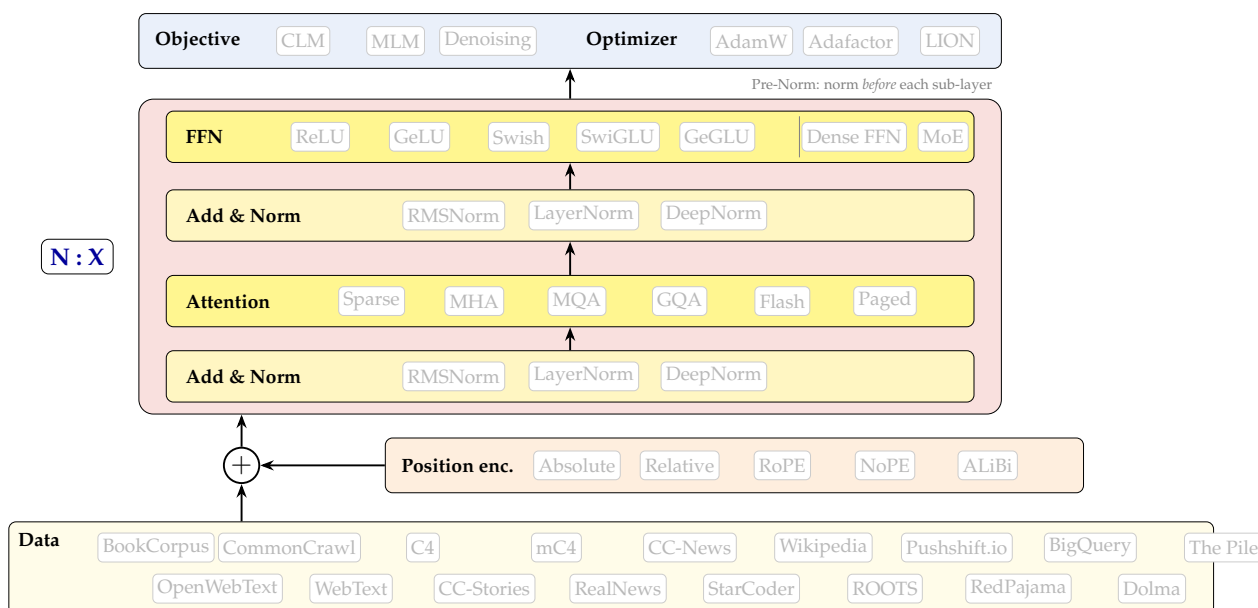


Section 4

The design axes of modern decoder-only LLMs

4.1 The unifying diagram

The lecture compresses *every* decoder-only LLM into one picture: a stack of N identical blocks, each with a choice of **normalisation**, **attention**, and **FFN/activation**; a **position encoding** added at the input; a **dataset mix** at the bottom; and an **objective** and **optimiser** at the top. A specific LLM is just **one selection per row** plus a value of N .



The unifying diagram with no choices highlighted. In Section 5 the same diagram is filled in for GPT-1, GPT-3 and PaLM.

Intuition : think of it as a menu

Each row is a menu. Choosing {CLM, AdamW, LayerNorm (post), GeLU dense FFN, MHA, absolute positions, BookCorpus, $N=12$ } gives GPT-1. Choosing {CLM, AdamW, RMSNorm (pre), SwiGLU, GQA, RoPE, $N=80$ } gives something close to LLaMA-2-70B. Most “new architectures” between 2019 and 2023 are new *selections*, not new *blocks*.

The rest of this section explains every option on every row. The slides *name* these options (the

lecture explores them in later weeks); the definitions, formulas and trade-offs below go beyond the slides, so treat them as a reference.

4.2 Objective: CLM, MLM, Denoising

Objective	Loss	Used by
CLM	$-\sum_t \log P(x_t x_{<t})$	GPT family, PaLM, LLaMA, OPT, BLOOM (all decoder-only)
MLM	$-\sum_{t \in \mathcal{M}} \log P(x_t \tilde{x})$	BERT, RoBERTa (encoders)
Denoising	$-\sum_j \log P(y_j y_{<j}, \tilde{x})$, where y = corrupted spans	T5, BART, UL2 (encoder-decoders)

For decoder-only LLMs the objective is almost always **CLM**: every token gives a gradient, and training matches generation exactly.

4.3 Optimiser: AdamW, Adafactor, LION

4.3.1 Adam and AdamW

With gradient g_t , learning rate η , $\beta_1, \beta_2 \in [0, 1)$:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, \end{aligned}$$

$$\text{Adam: } \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad \text{AdamW: } \theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right).$$

Intuition : what “decoupled” weight decay means

In Adam with L2 regularisation, the decay term $\lambda \theta$ is added to the *gradient* and then divided by $\sqrt{\hat{v}}$, so parameters with large gradient variance are barely regularised. AdamW applies the decay *directly* to the weights, outside the adaptive scaling, so every weight shrinks at the same relative rate. This generalises better and is the default for most LLMs (GPT-3 uses $\beta_2 = 0.95$; LLaMA uses AdamW with $\lambda = 0.1$).

Memory: 2 extra fp32 values per parameter (m, v). A 7B model needs $7 \times 10^9 \times 2 \times 4 \text{ B} = 56\text{GB}$ just for optimiser state.

4.3.2 Adafactor (recap from Week 7)

Factorises v for a matrix $n \times m$ into row and column statistics ($n + m$ numbers instead of nm) and can drop m . Used by T5 and PaLM.

4.3.3 LION (EvoLved Sign Momentum)

Discovered by automated program search (Chen et al., 2023):

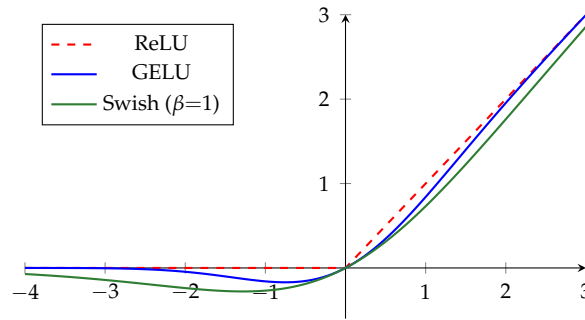
$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_t = \theta_{t-1} - \eta (\text{sign}(c_t) + \lambda \theta_{t-1}), \quad m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

- Tracks only **one** state (m), so it needs half of AdamW's optimiser memory.
- The update is a **sign**: every coordinate moves by exactly $\pm\eta$. This needs a smaller learning rate (typically 3–10 $\times$ smaller than AdamW) and larger weight decay.

4.4 Activation functions and the FFN

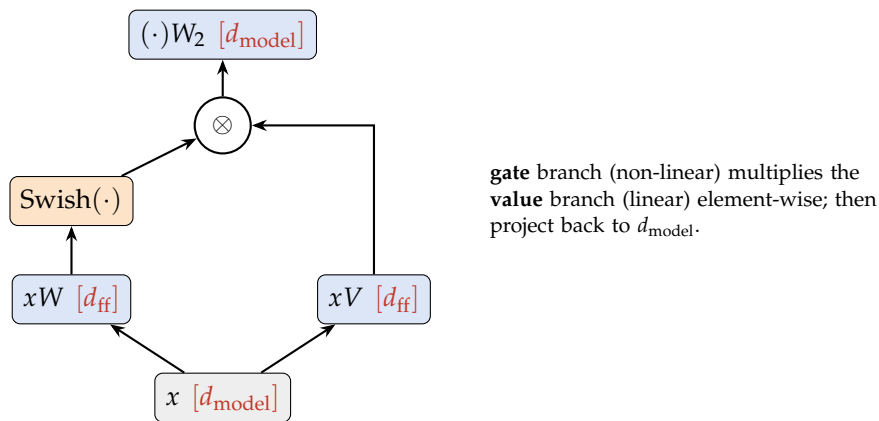
The position-wise FFN is $\text{FFN}(x) = f(xW_1 + b_1)W_2 + b_2$, with $W_1 [d_{\text{model}} \times d_{\text{ff}}]$ and $W_2 [d_{\text{ff}} \times d_{\text{model}}]$. The choice of f :

Name	Definition	Used in
ReLU	$\max(0, x)$	original Transformer, T5, OPT
GELU	$x \Phi(x)$	BERT, GPT-1/2/3, BLOOM
Swish / SiLU	$x \sigma(\beta x)$ ($\beta = 1$: SiLU)	(inside SwiGLU)
GLU	$\sigma(xW) \otimes (xV)$	—
GeGLU	$\text{GELU}(xW) \otimes (xV)$	T5 v1.1, Gemma
SwiGLU	$\text{Swish}(xW) \otimes (xV)$	PaLM, LLaMA, Mistral



GELU and Swish are smooth and non-monotonic (a small negative dip near $x \approx -1$), unlike ReLU's hard corner at 0.

4.4.1 Gated linear units (GLU family)



$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW) \otimes xV)W_2, \quad W, V [d_{\text{model}} \times d_{\text{ff}}], W_2 [d_{\text{ff}} \times d_{\text{model}}].$$

Derivation : why SwiGLU models use $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$

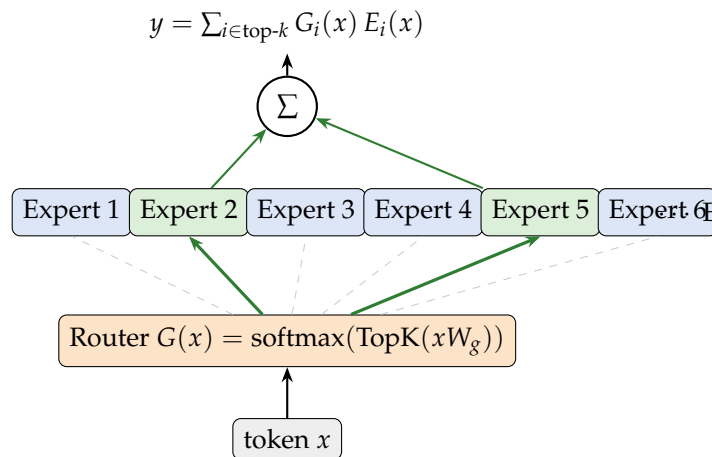
Standard FFN with $d_{\text{ff}} = 4d_{\text{model}}$: two matrices, $2 \times d_{\text{model}} \times 4d_{\text{model}} = 8d_{\text{model}}^2$ parameters.
 SwiGLU has *three* matrices (W, V, W_2): $3d_{\text{model}}d_{\text{ff}}$. To keep the parameter count equal,
 $3d_{\text{model}}d_{\text{ff}} = 8d_{\text{model}}^2 \Rightarrow d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$.

For LLaMA-7B, $d_{\text{model}} = 4096$: $\frac{8}{3} \cdot 4096 \approx 10,923$, rounded up to a multiple of 256:
 $d_{\text{ff}} = 11,008$.

(PaLM chose *not* to shrink d_{ff} : it kept $d_{\text{ff}} = 4d_{\text{model}}$ and accepted $1.5\times$ the FFN parameters.)

Intuition : why gating helps

The gate lets the network decide, per dimension and per token, how much of each feature passes through, a multiplicative interaction a plain activation cannot express. Empirically (Shazeer, 2020) GLU variants give lower perplexity at the same parameter count, and SwiGLU/GeGLU were the best.

4.5 Dense FFN vs. Mixture of Experts (MoE)

- Replace the single FFN with E expert FFNs and a learned **router**. Each token is sent to its top- k experts ($k = 1$ in Switch Transformer, $k = 2$ in GLaM and Mixtral).
- **Total parameters** grow with E , but **active parameters per token** grow only with k , so capacity rises while FLOPs per token stay nearly constant.
- A **load-balancing loss** stops the router from sending everything to a few experts: $\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^E f_i P_i$, where f_i is the fraction of tokens routed to expert i and P_i the mean router probability for it (minimised when both are uniform).

Derivation : GLaM's numbers

GLaM: 64 experts, top-2 routing, 1.2T total parameters, $\approx 97\text{B}$ active per token. Active fraction = $97/1200 \approx 8\%$. Inference FLOPs per token $\approx 2 \times 97\text{B}$, about half of GPT-3's $2 \times 175\text{B}$, while GLaM had more capacity. The cost moves from compute to *memory* (all 1.2T weights must be stored) and communication (routing tokens across devices).

4.6 Normalisation: type and placement

4.6.1 Types

For a token vector $x \in \mathbb{R}^d$:

$$\text{LayerNorm: } y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_i x_i, \quad \sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2,$$

$$\text{RMSNorm: } y = \frac{x}{\text{RMS}(x)} \odot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}.$$

Intuition : why RMSNorm

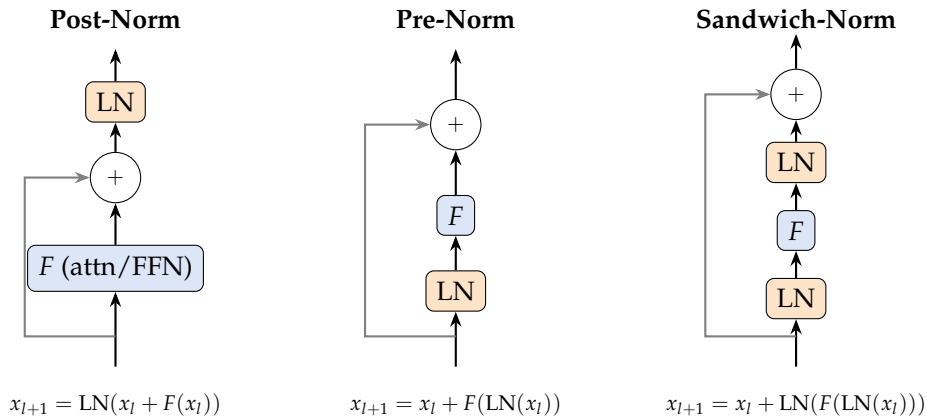
LayerNorm does two things: *re-centring* (subtract μ) and *re-scaling* (divide by σ). RMSNorm's authors found the re-scaling does most of the work. Dropping the mean (and β) saves computation (7–64% faster in their experiments) with no loss of quality. Used in Gopher, LLaMA and Mistral.

DeepNorm (DeepNet, 2022) modifies Post-Norm so very deep transformers (1000 layers) train stably:

$$x_{l+1} = \text{LN}(\alpha x_l + F(x_l)), \quad \alpha > 1,$$

and sub-layer weights are initialised scaled down by $\beta < 1$. For a decoder-only model with N layers, $\alpha = (2N)^{1/4}$ and $\beta = (8N)^{-1/4}$. Up-weighting the residual bounds how much each update can change the output, which prevents the “exploding update” problem of deep Post-Norm (used in GLM-130B).

4.6.2 Placement: Post-Norm, Pre-Norm, Sandwich



Derivation : why Pre-Norm trains more stably

Unroll Pre-Norm over L layers: $x_L = x_0 + \sum_{l=0}^{L-1} F_l(\text{LN}(x_l))$. The gradient contains an identity path:

$$\frac{\partial x_L}{\partial x_0} = I + \sum_l \frac{\partial F_l(\text{LN}(x_l))}{\partial x_0},$$

so gradients reach early layers undiminished regardless of depth. In Post-Norm every layer's output passes through LN, so the residual path is repeatedly re-normalised. There is no clean identity path, gradients near the output are large, and training needs careful learning-rate warm-up. Trade-off: Pre-Norm can lose some representational “depth” (later

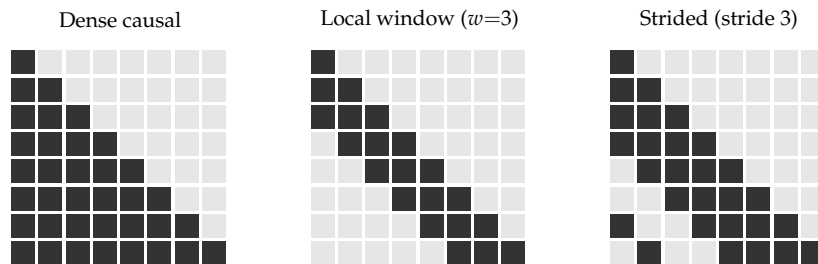
layers add ever-smaller relative updates), which motivates variants such as Sandwich-Norm (extra LN on the sub-layer output; CogView) and DeepNorm (fixes Post-Norm instead).

Model	Normalisation
Original Transformer, BERT, GPT-1	Post-Norm LayerNorm
GPT-2, GPT-3, PaLM, OPT, BLOOM	Pre-Norm LayerNorm
Gopher, LLaMA, Mistral	Pre-Norm RMSNorm
GLM-130B	DeepNorm

4.7 Attention variants

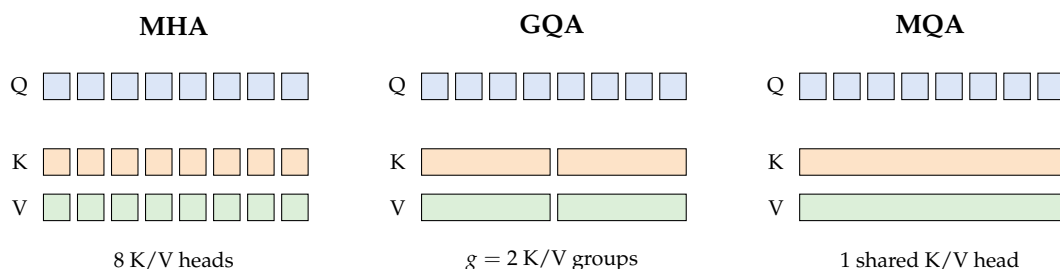
4.7.1 Full vs. sparse attention

Full (dense) causal attention costs $O(n^2)$ time and memory in the sequence length n . Sparse attention allows only a structured subset of (query, key) pairs.



- **Local / banded**: each token attends to the previous w tokens: $O(nw)$.
- **Strided** (Sparse Transformer): local window plus every s -th earlier token. With $s \approx \sqrt{n}$ the cost is $O(n\sqrt{n})$, and information can travel between any two positions in two hops.
- **GPT-3** alternates **dense** and **locally banded sparse** attention layers (this is why “Sparse” is highlighted for GPT-3 in the lecture).

4.7.2 MHA, MQA, GQA



- **MHA (multi-head)**: every query head has its own K and V heads.
- **MQA (multi-query)**: all query heads **share one** K and V head. Much smaller KV cache and faster decoding, with a small quality drop (PaLM, Falcon).

- **GQA (grouped-query):** query heads are split into g groups; each group shares one K/V head. It interpolates between the two ($g = h$ is MHA, $g = 1$ is MQA). Used in LLaMA-2-70B and Mistral.

Derivation : KV-cache size and why MQA/GQA matter

At inference, each generated token's keys and values are cached for every layer. Per token:

$$\text{KV bytes/token} = 2 \times L \times h_{kv} \times d_{\text{head}} \times (\text{bytes per value}),$$

where the 2 counts K and V. For LLaMA-2-70B ($L = 80, h = 64, d_{\text{head}} = 128, \text{fp16} = 2 \text{ bytes}$):

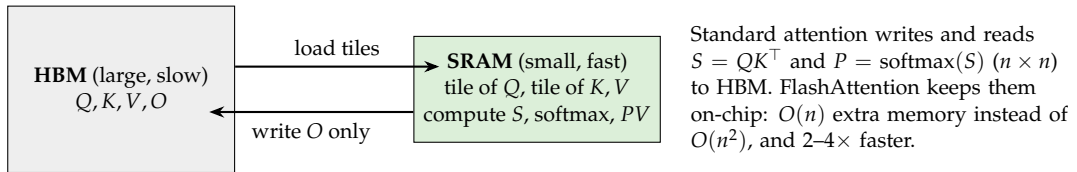
MHA ($h_{kv}=64$) : $2 \cdot 80 \cdot 64 \cdot 128 \cdot 2 = 2.62\text{MB/token} \Rightarrow 10\text{GiB}$ for a 4096-token sequence;

GQA ($h_{kv}=8$) : $327,680 \text{ B/token} \Rightarrow 1.25\text{GiB}$; MQA ($h_{kv}=1$) : $40,960 \text{ B/token} \Rightarrow 0.16\text{GiB}$.

Decoding is *memory-bandwidth bound*: every step re-reads the whole cache. Shrinking the cache $8\times$ (GQA) speeds decoding and allows bigger batches.

4.7.3 FlashAttention

FlashAttention computes *exact* attention but is **IO-aware**. It never writes the $n \times n$ score matrix to slow GPU HBM; it processes Q, K, V in tiles that fit in fast on-chip SRAM.



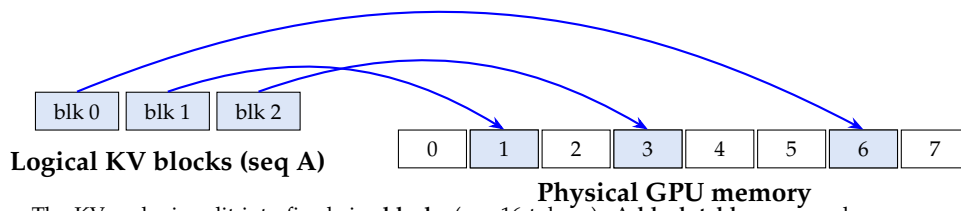
Derivation : the online-softmax trick that makes tiling possible

Softmax over a row needs the maximum and the sum over *all* keys, but we see the keys one tile at a time. Keep running statistics: max m , normaliser ℓ and an unnormalised output $\tilde{o}$. For a new tile with scores s_j and values v_j :

$$m' = \max(m, \max_j s_j), \quad \ell' = e^{m-m'} \ell + \sum_j e^{s_j-m'}, \quad \tilde{o}' = e^{m-m'} \tilde{o} + \sum_j e^{s_j-m'} v_j.$$

After the last tile, $o = \tilde{o} / \ell$. The factor $e^{m-m'}$ rescales the old partial sums whenever a larger max appears, so the result equals exact softmax attention. The backward pass recomputes S tile by tile instead of storing it (recomputation is cheaper than memory traffic).

4.7.4 PagedAttention



The KV cache is split into fixed-size **blocks** (e.g. 16 tokens). A **block table** maps each sequence's logical blocks to any free physical blocks, like virtual-memory pages in an OS. There is no need to reserve contiguous memory for the maximum length, fragmentation drops (vLLM reports under 4% waste), and sequences can **share** blocks (parallel sampling, beam search) with copy-on-write.

Key takeaway

Sparse attention changes *what* is computed (an approximation). **MQA/GQA** change the *parameterisation* (fewer K/V heads). **Flash** changes *how* exact attention is computed (IO-aware kernels). **Paged** changes *how the KV cache is stored* at serving time. They are complementary; modern systems combine GQA + Flash + Paged.

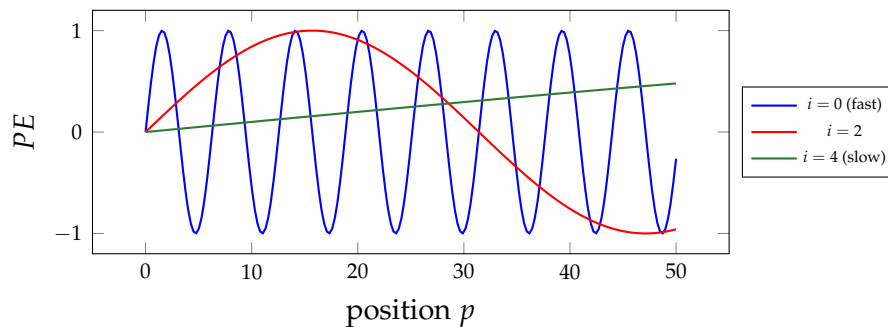
4.8 Position encodings

Self-attention is **permutation-equivariant**: without position information, “dog bites man” and “man bites dog” look the same (up to a causal mask). Position encodings inject order.

4.8.1 Absolute (sinusoidal or learned)

$$PE_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad PE_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d}}\right), \quad x_p \leftarrow x_p + PE_p [d].$$

Learned absolute embeddings (a trainable table $P [T_{\max} \times d]$) are used by BERT, GPT-1/2/3 and OPT. **Limitation**: the model has never seen positions beyond $T_{\max}$, so it does not extrapolate to longer contexts.



Each dimension pair is a sinusoid of a different wavelength (2π to $2\pi \cdot 10^4$), so position is written like a multi-frequency “clock”.

4.8.2 Relative position

Instead of encoding *where* a token is, encode *how far apart* two tokens are: add a learned term depending on $i - j$ to the attention logits. T5 uses a learned scalar bias per head for **bucketed**

distances (exact for small distances, log-spaced buckets for large ones):

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + b_h(\text{bucket}(i - j)).$$

4.8.3 RoPE: rotary position embedding

Rotate each 2-D pair of query/key coordinates by an angle proportional to the position.

$$R(m\theta_i) = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad \theta_i = 10000^{-2i/d}, \quad i = 0, \dots, \frac{d}{2} - 1,$$

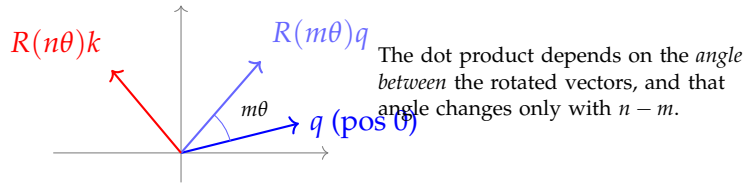
$$\tilde{q}_m = R_m q_m, \quad \tilde{k}_n = R_n k_n \quad (\text{block-diagonal } R \text{ applied pair by pair}).$$

Derivation : why RoPE encodes relative position

Rotation matrices are orthogonal and compose additively: $R(a)^\top = R(-a)$ and $R(a)R(b) = R(a + b)$. Then

$$\tilde{q}_m^\top \tilde{k}_n = q_m^\top R(m\theta)^\top R(n\theta) k_n = q_m^\top R((n - m)\theta) k_n.$$

The attention logit depends on positions **only through the offset** $n - m$: relative position, obtained from an absolute-style operation applied to q and k individually. Complex-number view: treating each pair as $z \in \mathbb{C}$, RoPE multiplies by $e^{im\theta}$, and $\text{Re}[(qe^{im\theta})(ke^{in\theta})] = \text{Re}[q\bar{k}e^{i(m-n)\theta}]$.



Used by GPT-J, PaLM, LLaMA and Mistral.

4.8.4 ALiBi: attention with linear biases

No position embeddings at all. Instead add a head-specific linear penalty on distance to the attention logits:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} - m_h(i - j), \quad j \leq i,$$

with fixed (not learned) slopes forming a geometric sequence. For H heads, $m_h = 2^{-8h/H}$; for $H = 8$: $\frac{1}{2}, \frac{1}{4}, \dots, \frac{1}{256}$.

-0				
-1	-0			
-2	-1	-0		
-3	-2	-1	-0	
-4	-3	-2	-1	-0

$\times m_h$ (head slope), added to $q_i^\top k_j$

Nearby tokens get a small penalty and distant ones a large one, a built-in recency bias. ALiBi **extrapolates** well: a model trained on 1024 tokens works on 2048+. Used by BLOOM and MPT.

4.8.5 NoPE: no position encoding

Decoder-only models with a **causal mask** can train with *no* explicit position encoding. The mask breaks the permutation symmetry: token i attends over exactly i tokens, so the model can infer absolute position (e.g. attention to a start token spreads over more keys at later positions). Some studies found NoPE generalises to longer lengths better than absolute embeddings.

Method	Where	Extrapolation	Examples
Absolute (learned/sin.)	added to input embeddings	poor	GPT-1/2/3, BERT, OPT
Relative (bias) RoPE	added to logits, by $i - j$ rotates q, k	moderate moderate (extend- able)	T5, Gopher GPT-J, PaLM, LLaMA
ALiBi	linear distance bias on logits	good	BLOOM, MPT
NoPE	none (causal mask only)	reported good	research models

4.9 Depth and training steps

N (number of blocks) and the number of training steps/tokens are the remaining knobs: GPT-1 has $N = 12$, GPT-3 $N = 96$, PaLM $N = 118$. Chinchilla-style reasoning (Section 2) decides how many tokens to train on for a given size.

4.10 Datasets

Common Crawl is the source for most pre-training datasets; the rest are curated domains (books, Wikipedia, news, code, dialogue).

Dataset	Description
BookCorpus	≈11k unpublished books (≈800M words); GPT-1, BERT.
Common Crawl	Monthly petabyte-scale web scrapes since 2008; raw material for C4, mC4, RefinedWeb, RedPajama, Dolma, ...
C4 / mC4	Cleaned English Common Crawl (T5, 750GB) / multilingual version covering 101 languages (mT5).
WebText / OpenWebText	Reddit-linked pages with ≥ 3 karma (GPT-2, 40GB) / an open re-creation of it.
CC-News / RealNews	News articles from Common Crawl (CC-News: RoBERTa) / news from Google-News-indexed domains (Grover, 120GB).
CC-Stories	Common Crawl subset selected to resemble Winograd-schema story-like text (31GB); used in RoBERTa, OPT.
Wikipedia	Encyclopedic text; in almost every mix.
Pushshift.io	Reddit dumps (comments, submissions); used in OPT and dialogue models.
BigQuery	Public GitHub code via Google BigQuery; used for code models (e.g. CodeGen).
StarCoder data	The Stack: permissively licensed source code, 80+ languages (StarCoder).
The Pile	825GB, 22 diverse subsets (academic, code, books, web) from EleutherAI; GPT-J, GPT-NeoX.
ROOTS	1.6TB, 46 natural and 13 programming languages; BLOOM.
RedPajama	Open reproduction of the LLaMA training mix (≈1.2T tokens).
Dolma	AI2's 3T-token open corpus with documented pipeline; OLMo.

Note on the slides

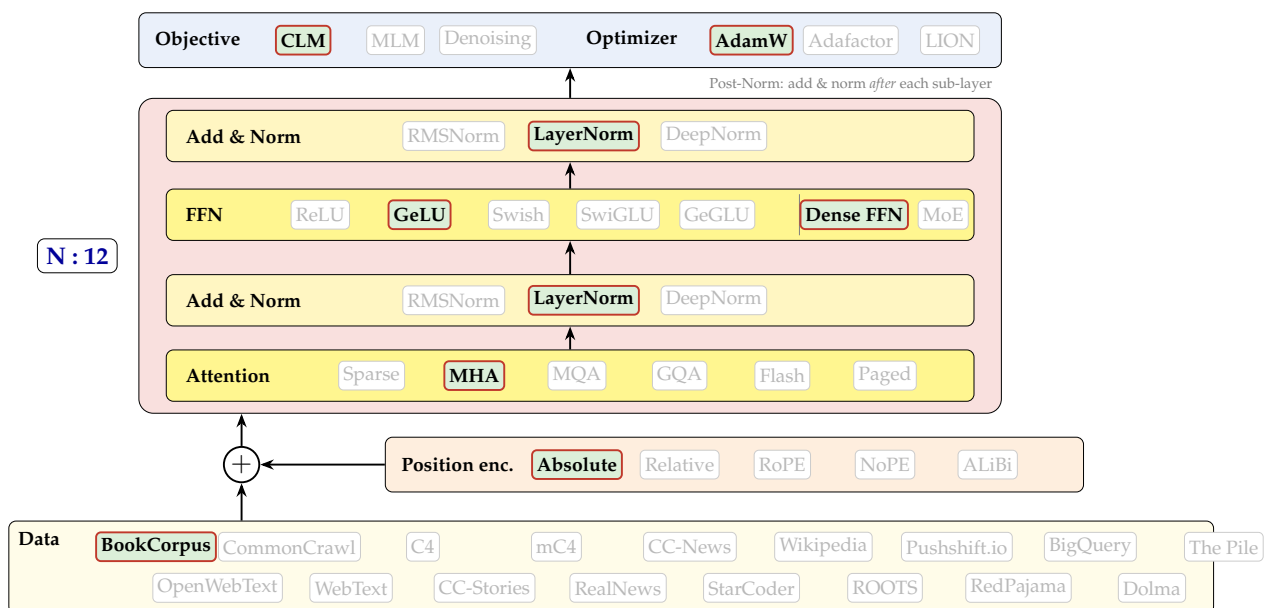
In the PaLM diagram some datasets carry an asterisk (CC-News*, RealNews*, StarCoder*). The asterisk means “a dataset *like* this”: PaLM’s actual mix (social-media conversations, filtered web pages, books, GitHub, Wikipedia, news) is proprietary, and the slide maps it onto the closest public datasets.

Section 5

Case studies: GPT-1, GPT-3 175B, PaLM 540B

The lecture fills in the unifying diagram for three landmark decoder-only models. Highlighted (green) chips are the choices each model made *as marked on the slides*.

5.1 GPT-1 (2018)



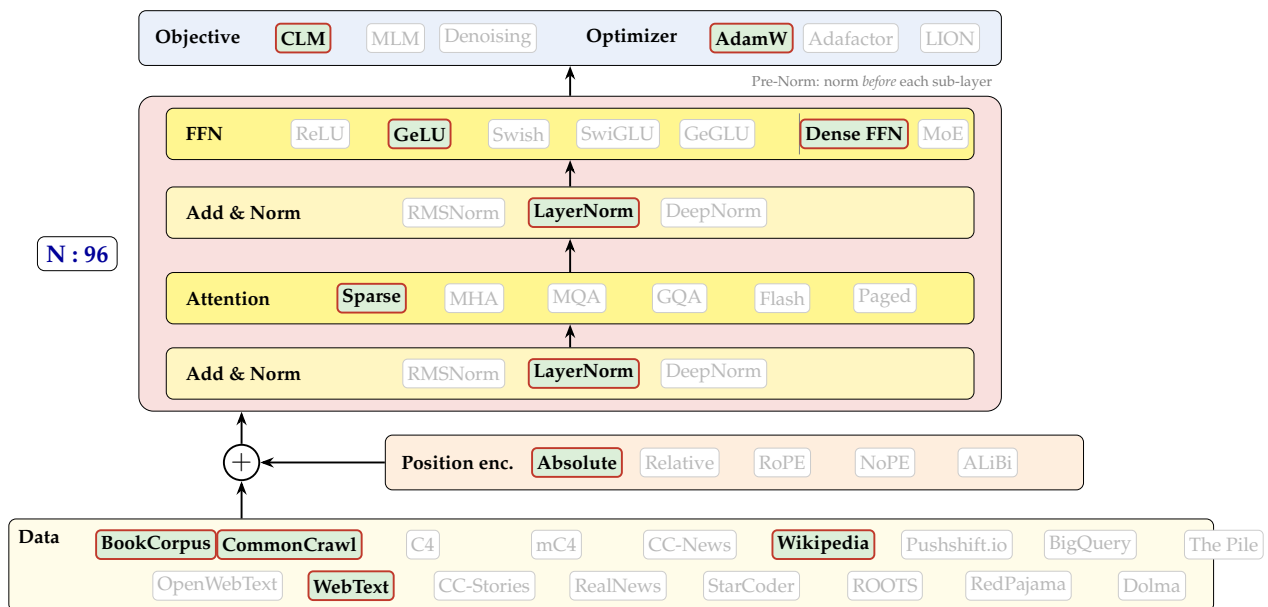
Objective	CLM	Layers N	12
Optimiser	Adam(W) (max LR 2.5×10^{-4})	d_{model} / heads	768 / 12
Norm	LayerNorm, Post-Norm	d_{ff}	3072
Activation	GELU, dense FFN	Context	512 tokens
Attention	masked MHA	Vocab	40,478 (BPE, 40k merges)
Positions	learned absolute	Data	BookCorpus ($\approx 7\text{k}$ books)

Derivation : GPT-1 parameter count

$12Ld_{\text{model}}^2 = 12 \cdot 12 \cdot 768^2 = 84.9\text{M}$; token embeddings $40478 \cdot 768 = 31.1\text{M}$; positions $512 \cdot 768 = 0.4\text{M}$. Total $\approx 116.4\text{M} \approx 117\text{M} \checkmark$.

Note on the slides

The slide highlights “AdamW” for GPT-1. The paper describes Adam with a modified L2 regularisation ($w = 0.01$ on non-bias weights), which is essentially decoupled weight decay, so AdamW is a fair label.

5.2 GPT-3 175B (2020)

Objective	CLM	Layers N	96
Optimiser	Adam(W), $\beta_1=0.9, \beta_2=0.95$	d_{model} / heads / d_{head}	12288 / 96 / 128
Norm	LayerNorm, Pre-Norm ("Norm & add")	d_{ff}	$4d_{\text{model}} = 49152$
Activation	GELU, dense FFN	Context	2048 tokens
Attention	alternating dense and locally banded sparse	Batch	3.2M tokens
Positions	learned absolute	Training tokens	300B

Data mix (tokens sampled during training): Common Crawl, filtered (60%), WebText2 (22%), Books1 (8%), Books2 (8%), Wikipedia (3%). The slide marks these as CC, WebText, BookCorpus, Wikipedia. High-quality sources were *up-sampled*: Wikipedia was seen about 3.4 times, while only 0.44 epochs of Common Crawl were used.

Derivation : GPT-3 parameter count

$12Ld_{\text{model}}^2 = 12 \cdot 96 \cdot 12288^2 = 173.95\text{B}$; token embeddings $50257 \cdot 12288 = 0.62\text{B}$; positions $2048 \cdot 12288 = 0.03\text{B}$. Total $\approx 174.6\text{B} \approx \mathbf{175B}$ ✓.

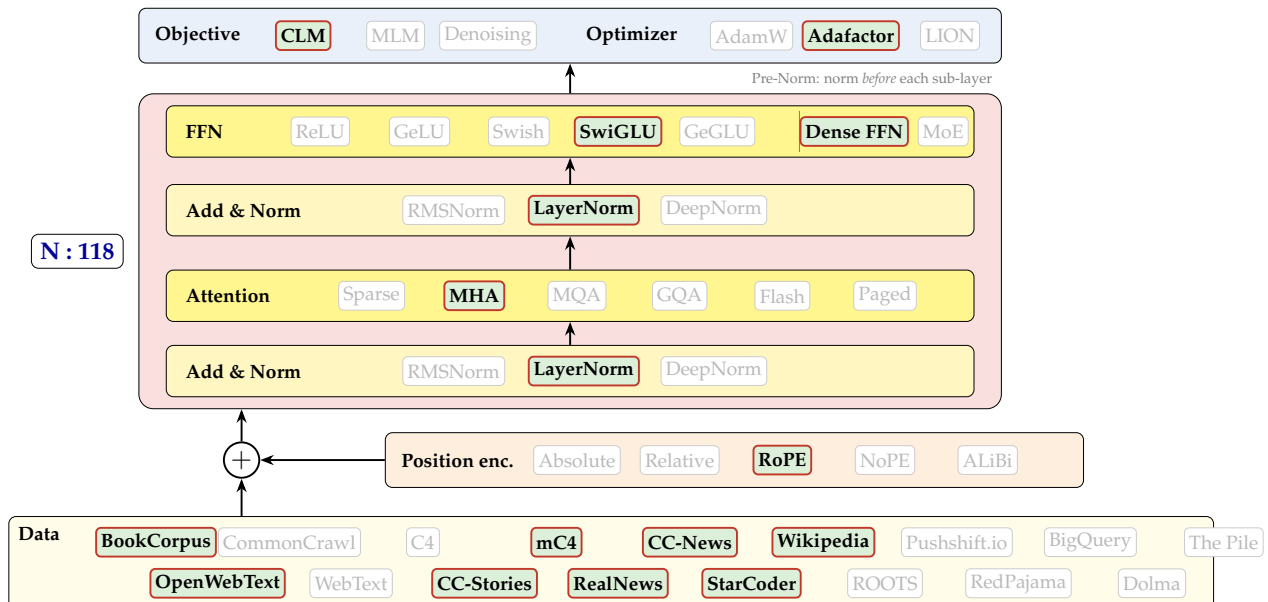
Training compute: $6ND = 6 \cdot 175 \times 10^9 \cdot 300 \times 10^9 \approx 3.15 \times 10^{23}$ FLOPs.

Intuition : what changed from GPT-1 to GPT-3

The *menu choices* barely changed (CLM, GELU, LayerNorm, absolute positions). What changed: **Pre-Norm** (needed for stable training at 96 layers), **sparse attention layers** (cheaper long context), and above all **scale**: N from 12 to 96, d_{model} from 768 to 12288, data from about 1B words of books to 300B tokens of mostly web text. Scale alone produced few-shot in-context

learning.

5.3 PaLM 540B (2022)



Objective	CLM	Layers N	118
Optimiser	Adafactor (unfactorised)	$d_{\text{model}} / \text{heads} / d_{\text{head}}$	18432 / 48 / 256
Norm	LayerNorm (no biases)	d_{ff}	$4d_{\text{model}} = 73728$
Activation	SwiGLU	Context	2048 tokens
Attention	multi-head in the slide (see note)	Vocab	256k SentencePiece
Positions	RoPE	Training tokens	780B

Data (780B tokens): social-media conversations (50%), filtered web pages (27%), books (13%), GitHub code (5%), Wikipedia (4%), news (1%). The slide maps this onto public look-alikes (mC4, CC-News*, RealNews*, StarCoder*, ...).

Note on the slides

Two details differ from the PaLM paper:

- The slide highlights **MHA**; the paper uses **multi-query attention** (one shared K/V head per layer, $d_{\text{head}} = 256$), a key reason it serves efficiently. In an exam, follow the course's marking but know the paper's detail.
- PaLM uses a **parallel** block, $y = x + \text{MLP}(\text{LN}(x)) + \text{Attn}(\text{LN}(x))$, instead of the serial attention→FFN drawn in the diagram. It is about 15% faster at large scale.

Derivation : PaLM parameter count (with MQA and SwiGLU)

Per layer:

- $W_Q: d_{\text{model}} \times (h d_{\text{head}}) = 18432 \times 12288$; W_O the same: $2 \times 226.5\text{M} = 453.0\text{M}$.
- W_K, W_V (one shared head each): $2 \times 18432 \times 256 = 9.4\text{M}$.
- SwiGLU FFN: $3 \times d_{\text{model}} \times d_{\text{ff}} = 3 \times 18432 \times 73728 = 4,076.9\text{M}$.

Per layer $\approx 4.539\text{B}$; $\times 118$ layers = 535.6B. Embeddings (shared input/output) $256000 \times 18432 = 4.7\text{B}$. Total $\approx 540.4\text{B}$ ✓.

Note the FFN is about 90% of the per-layer parameters.

5.4 Side-by-side

	GPT-1	GPT-3 175B	PaLM 540B
Layers N	12	96	118
d_{model}	768	12288	18432
Norm (type, placement)	LayerNorm, post	LayerNorm, pre	LayerNorm, pre (parallel block)
Activation / FFN	GELU, dense	GELU, dense	SwiGLU, dense
Attention	MHA	dense + sparse	MHA (slide) / MQA (paper)
Positions	absolute	absolute	RoPE
Optimiser	Adam(W)	Adam(W)	Adafactor
Data	BookCorpus	CC, WebText2, Books, Wiki	web, conversations, books, code, Wiki, news
Tokens	$\approx 1\text{B}$ words (100 epochs)	300B	780B
Tokens / parameter	–	1.7	1.4

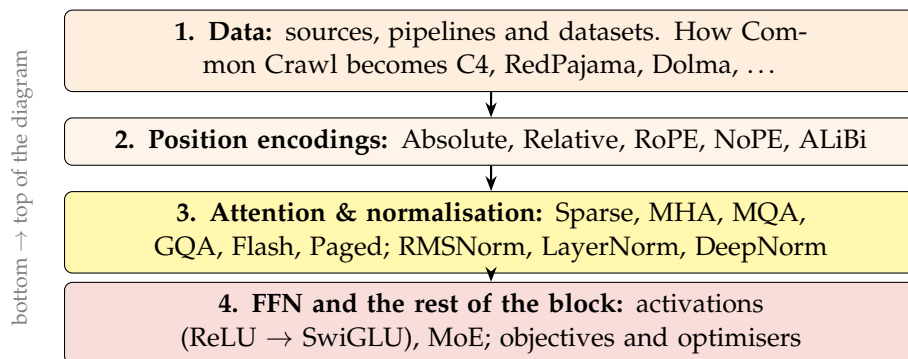
Key takeaway

Over four years the *block* evolved: post-norm $\rightarrow$ pre-norm; GELU $\rightarrow$ SwiGLU; absolute $\rightarrow$ RoPE; MHA $\rightarrow$ MQA/GQA; Adam $\rightarrow$ Adafactor/AdamW with tuned β_2 . The *recipe* (decoder-only, CLM, next-token prediction) stayed the same. By Chinchilla's criterion both GPT-3 and PaLM are under-trained (far fewer than 20 tokens per parameter).

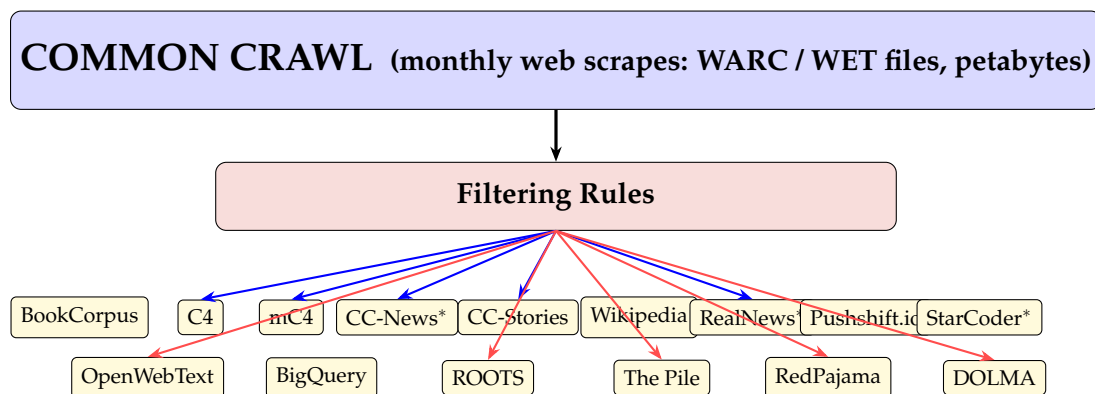
Section 6

The road ahead

The lecture ends by laying out the rest of the course as a walk through the unifying diagram, row by row, starting at the bottom.

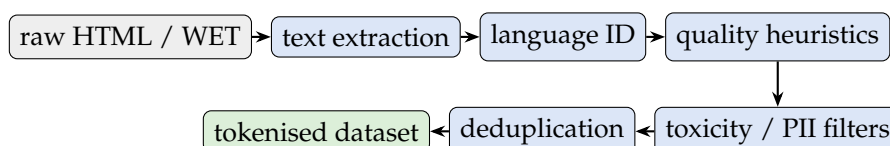


6.1 Data sources, data pipelines, datasets



Most of these datasets are *Common Crawl* + a set of *filtering rules* (in whole or in part). The rest (books, Wikipedia, Reddit dumps, GitHub/BigQuery code) come from curated sources.

6.1.1 What “filtering rules” typically include (preview)



- **Language identification**, e.g. keep pages classified as English with probability ≥ 0.99 (C4).
- **Quality heuristics**: minimum words and sentences, terminal punctuation, symbol-to-word ratio, removing boilerplate, or a classifier trained to recognise “high-quality” reference text (as for GPT-3).
- **Deduplication**: exact (hashes) and fuzzy (MinHash / LSH) at document, paragraph or n-gram level. Recall from Week 7 that repeated data leads to memorisation.
- **Safety**: bad-word lists, toxicity classifiers, PII removal.

Key takeaway

Week 7 showed that data *quality* matters as much as quantity (unfiltered C4 was worse despite being $8\times$ larger). The road ahead begins with how modern pipelines turn Common Crawl into high-quality pre-training corpora, then moves up the diagram: position encodings, attention variants, normalisation, activations and MoE.

Part A

Revision sheet

Key takeaway : the lecture in eight lines

1. Group LLMs by architecture: **encoder-only**, **encoder-decoder**, **decoder-only** (the evolutionary tree).
2. Encoder-only (BERT family) dominated briefly; GPT-2 argued for decoder-only, T5 for encoder-decoder.
3. **Scaling laws** (model vs. data, and in what proportion) triggered a race: GPT-3 175B, GPT-J 6B, Jurassic-1 178B, Gopher 280B, GLaM 1.2T (MoE).
4. **Decoder-only models dominated**: OPT-175B (open), InstructGPT (RLHF alignment) → ChatGPT; the number of open and closed LLMs keeps growing.
5. Even within decoder-only there are dozens of models. They differ in (a) the **dataset** (mostly Common Crawl-derived) and (b) **design choices** that cut compute or raise performance.
6. **Unifying diagram** rows: Objective (CLM/MLM/Denoising), Optimiser (AdamW/Adafactor/LION), Norm (Layer/RMS/Deep; post/pre/sandwich), FFN (ReLU/Gelu/Swish/SwiGLU/GeGLU; dense/MoE), Attention (sparse/MHA/MQA/GQA/Flash/Paged), Position (absolute/relative/RoPE/NoPE/ALiBi), Data, and N .
7. Case studies: **GPT-1** ($N=12$, post-LN, GeLU, MHA, absolute, BookCorpus, AdamW); **GPT-3** ($N=96$, pre-LN, GeLU, sparse, absolute, CC/WebText/Books/Wiki, AdamW); **PaLM** ($N=118$, pre-LN, SwiGLU, MHA on the slide (MQA in the paper), RoPE, Adafactor, mC4-like mix).
8. **Road ahead**: data sources → pipelines → filtering rules → datasets, then position encodings, attention, normalisation, activations/MoE.

A.1 Formula sheet

Concept	Formula
Kaplan scaling	$L(N) = (N_c/N)^{0.076}, L(D) = (D_c/D)^{0.095}; N_{\text{opt}} \propto C^{0.73}$
Chinchilla loss	$L(N, D) = E + A/N^\alpha + B/D^\beta, E=1.69, A=406.4, B=410.7, \alpha=0.34, \beta=0.28$
Compute	$C \approx 6ND$; optimal $N \propto C^{\beta/(\alpha+\beta)}, D \approx 20N$, so $N = \sqrt{C/120}$
RLHF reward loss	$-\log \sigma(r(x, y_w) - r(x, y_l)); \text{RL: } \max \mathbb{E}[r] - \beta \text{KL}(\pi \ \pi_{\text{SFT}})$
AdamW	$\theta \leftarrow \theta - \eta (\hat{m} / (\sqrt{\hat{v}} + \epsilon) + \lambda \theta)$
LION	$\theta \leftarrow \theta - \eta (\text{sign}(\beta_1 m + (1 - \beta_1)g) + \lambda \theta); m \leftarrow \beta_2 m + (1 - \beta_2)g$
Swish / SwiGLU	$x\sigma(\beta x); (\text{Swish}(xW) \otimes xV)W_2, d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ for equal params
MoE	$y = \sum_{i \in \text{top-}k} G_i(x)E_i(x); \text{aux loss } \alpha E \sum_i f_i P_i$
LayerNorm / RM-SNorm	$\frac{x-\mu}{\sigma}\gamma + \beta / \frac{x}{\sqrt{\frac{1}{d}\sum x_i^2}}\gamma$
Post / Pre / Sandwich	$\text{LN}(x + F(x)) / x + F(\text{LN}(x)) / x + \text{LN}(F(\text{LN}(x)))$
DeepNorm	$\text{LN}(\alpha x + F(x)), \text{decoder: } \alpha = (2N)^{1/4}, \text{init scale } \beta = (8N)^{-1/4}$
KV cache	$2 \cdot L \cdot h_{kv} \cdot d_{\text{head}} \cdot \text{bytes per token}$
Online softmax	$m' = \max(m, \max s), \ell' = e^{m-m'}\ell + \sum e^{s-m'}, \tilde{o}' = e^{m-m'}\tilde{o} + \sum e^{s-m'}v$
Sinusoidal PE	$\sin(p/10000^{2i/d}), \cos(p/10000^{2i/d})$
RoPE	$\theta_i = 10000^{-2i/d}; (R_m q)^\top (R_n k) = q^\top R_{n-m} k$
ALiBi	$s_{ij} = q_i^\top k_j / \sqrt{d_k} - m_h(i - j), m_h = 2^{-8h/H}$
Params (dense dec.)	$\approx 12Ld_{\text{model}}^2 + Vd_{\text{model}}$

A.2 Which model chose what

Model	Norm	Activation	Attention	Position	Optimiser
GPT-1	LN post	GELU	MHA	learned abs.	Adam(W)
GPT-3	LN pre	GELU	dense+sparse	learned abs.	Adam(W)
Gopher	RMSNorm pre	–	MHA	relative	Adam
PaLM	LN pre (parallel)	SwiGLU	MQA (paper)	RoPE	Adafactor
BLOOM	LN pre	GELU	MHA	ALiBi	Adam
LLaMA / LLaMA-2	RMSNorm pre	SwiGLU	MHA / GQA (70B)	RoPE	AdamW
GLM-130B	DeepNorm	GeGLU	MHA	RoPE	AdamW

Part B

Numerical problems with worked solutions

Problem 1 (Scaling laws: what a doubling buys)

Under Kaplan's $L(N) = (N_c/N)^{0.076}$: (a) by what factor does the loss change when N doubles? (b) By what factor must N grow to halve the loss?

Solution

(a) $L(2N)/L(N) = 2^{-0.076} \approx 0.9487$, a $\approx 5.1\%$ reduction per doubling.
(b) Need $(N'/N)^{-0.076} = \frac{1}{2} \Rightarrow N'/N = 2^{1/0.076} = 2^{13.16} \approx 9,140\times$. Power laws give steady but expensive gains.

Problem 2 (Training compute $C \approx 6ND$)

Compute training FLOPs for GPT-3 (175B, 300B tokens), Chinchilla (70B, 1.4T) and PaLM (540B, 780B).

Solution

GPT-3: $6 \cdot 1.75 \times 10^{11} \cdot 3 \times 10^{11} = 3.15 \times 10^{23}$.
Chinchilla: $6 \cdot 7 \times 10^{10} \cdot 1.4 \times 10^{12} = 5.88 \times 10^{23}$.
PaLM: $6 \cdot 5.4 \times 10^{11} \cdot 7.8 \times 10^{11} = 2.53 \times 10^{24}$ (about $8\times$ GPT-3).

Problem 3 (Compute-optimal sizing)

With $D = 20N$ and $C = 6ND$, find the compute-optimal N and D for $C = 10^{24}$ FLOPs.

Solution

$C = 120N^2 \Rightarrow N = \sqrt{10^{24}/120} = \sqrt{8.33 \times 10^{21}} \approx 9.1 \times 10^{10}$ (≈ 91 B parameters); $D = 20N \approx 1.83 \times 10^{12}$ tokens.

Problem 4 (Chinchilla exponents)

Using $\alpha = 0.34$, $\beta = 0.28$, compute the exponents a, b in $N_{\text{opt}} \propto C^a$, $D_{\text{opt}} \propto C^b$. If compute grows $100\times$, how do N and D grow?

Solution

$a = \beta / (\alpha + \beta) = 0.28 / 0.62 = 0.452$; $b = \alpha / (\alpha + \beta) = 0.548$; $a + b = 1$ ✓ (since $C \propto ND$).
 $100 \times \text{compute}$: $N \times 100^{0.452} \approx 8.0 \times$, $D \times 100^{0.548} \approx 12.4 \times$. Under Kaplan (0.73/0.27): $N \times 28.8$, $D \times 3.5$, a very different allocation.

Problem 5 (Tokens per parameter)

Compute tokens/parameter for GPT-3, PaLM, LLaMA-7B (1T tokens) and LLaMA-2-70B (2T). Which are “under-trained” by the 20:1 rule?

Solution

GPT-3: $300 / 175 = 1.71$; PaLM: $780 / 540 = 1.44$ (both far below 20, so under-trained).
 LLaMA-7B: $1000 / 7 \approx 143$; LLaMA-2-70B: $2000 / 70 \approx 28.6$ (both beyond 20). These were deliberately “over-trained” small models, because inference cost scales with N , not D .

Problem 6 (Chinchilla loss comparison)

Using $L(N, D) = 1.69 + 406.4N^{-0.34} + 410.7D^{-0.28}$, compare GPT-3 (175B, 300B), Gopher (280B, 300B) and Chinchilla (70B, 1.4T).

Solution

GPT-3: ≈ 2.002 ; Gopher: ≈ 1.993 ; Chinchilla: ≈ 1.937 .
 For Chinchilla: model term $406.4 / (7 \times 10^{10})^{0.34} \approx 0.083$, data term $410.7 / (1.4 \times 10^{12})^{0.28} \approx 0.163$. The smaller model trained on more data has the **lowest** predicted loss, with the same compute as Gopher.

Problem 7 (GPT-3 parameter count)

Verify 175B using $12Ld_{\text{model}}^2 + Vd_{\text{model}} + Td_{\text{model}}$ with $L = 96$, $d_{\text{model}} = 12288$, $V = 50257$, $T = 2048$.

Solution

$12 \cdot 96 \cdot 12288^2 = 173,946,175,488$; $Vd_{\text{model}} = 617,558,016$; $Td_{\text{model}} = 25,165,824$. Total $\approx 174.59\text{B} \approx 175\text{B}$ ✓.

Problem 8 (PaLM parameter count)

PaLM: $L = 118$, $d_{\text{model}} = 18432$, $h = 48$, $d_{\text{head}} = 256$, multi-query attention, SwiGLU with $d_{\text{ff}} = 73728$, vocab 256k (shared embeddings). Estimate the total.

Solution

$W_Q + W_O = 2 \cdot 18432 \cdot 12288 = 452,984,832$; $W_K + W_V = 2 \cdot 18432 \cdot 256 = 9,437,184$; FFN $= 3 \cdot 18432 \cdot 73728 = 4,076,863,488$.
 Per layer $= 4,539,285,504$; $\times 118 = 535.6\text{B}$; $+ 256000 \cdot 18432 = 4.7\text{B} \Rightarrow \approx 540.4\text{B}$ ✓.

Problem 9 (SwiGLU hidden size)

For $d_{\text{model}} = 4096$ and $d_{\text{model}} = 5120$, find the SwiGLU d_{ff} that matches the parameters of a $4d_{\text{model}}$ ReLU FFN, rounded up to a multiple of 256.

Solution

$d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$. $d_{\text{model}} = 4096$: $10,922.7 \rightarrow \mathbf{11,008}$ (43×256), LLaMA-7B.

$d_{\text{model}} = 5120$: $13,653.3 \rightarrow \mathbf{13,824}$ (54×256), LLaMA-13B.

Check ($d_{\text{model}} = 4096$): ReLU FFN $2d_{\text{model}}(4d_{\text{model}}) = 134,217,728$; SwiGLU $3d_{\text{model}}d_{\text{ff}} = 3 \cdot 4096 \cdot 11008 = 135,266,304$, nearly equal.

Problem 10 (MoE parameter accounting)

A model has 32 layers; each FFN is replaced by $E = 8$ experts of 0.2B parameters each, with top-2 routing. The non-FFN parameters total 1.5B. Find the total and active parameters per token.

Solution

Expert parameters: $32 \cdot 8 \cdot 0.2 = 51.2\text{B}$; total $= 51.2 + 1.5 = 52.7\text{B}$.

Active: $32 \cdot 2 \cdot 0.2 = 12.8\text{B}$ plus the shared $1.5\text{B} = 14.3\text{B}$ (27% of total). FLOPs per token scale with 14.3B , but memory must hold 52.7B .

Problem 11 (KV cache: MHA vs. GQA vs. MQA)

LLaMA-2-70B: $L = 80$, $h = 64$, $d_{\text{head}} = 128$, fp16. Compute the KV cache per token and for a 4096-token sequence with $h_{kv} = 64$ (MHA), 8 (GQA), 1 (MQA).

Solution

Per token $= 2 \cdot L \cdot h_{kv} \cdot d_{\text{head}} \cdot 2 \text{ B}$.

MHA: $2 \cdot 80 \cdot 64 \cdot 128 \cdot 2 = 2,621,440 \text{ B}$ (2.5MiB) $\Rightarrow \times 4096 = 10\text{GiB}$.

GQA: $327,680 \text{ B} \Rightarrow 1.25\text{GiB}$. MQA: $40,960 \text{ B} \Rightarrow 0.156\text{GiB}$.

Problem 12 (KV cache of GPT-3)

GPT-3: $L = 96$, $h = 96$, $d_{\text{head}} = 128$, fp16, context 2048. KV cache for one full-length sequence? How many such sequences fit in 80GB after the weights (350GB in fp16) are spread over 8 GPUs?

Solution

Per token $= 2 \cdot 96 \cdot 96 \cdot 128 \cdot 2 = 4,718,592 \text{ B} = 4.5\text{MiB}$; $\times 2048 = 9\text{GiB}$ per sequence.

8 GPUs $\times$ 80GB $= 640\text{GB}$; minus 350GB of weights leaves $\approx 290\text{GB} \Rightarrow \approx 30$ concurrent sequences (ignoring activations). This is why MQA/GQA and PagedAttention matter for serving.

Problem 13 (RoPE angles and the relative property)

(a) For $d = 8$, list $\theta_i = 10000^{-2i/d}$ and the rotation angles at position $m = 3$. (b) Take 2-D vectors $q = k = (1, 0)$ and $\theta = 0.5$. Compute $(R_m q)^\top (R_n k)$ for $(m, n) = (2, 5)$ and $(12, 15)$.

Solution

(a) $\theta = (1, 0.1, 0.01, 0.001)$; angles at $m = 3$: $(3, 0.3, 0.03, 0.003)$ radians. Low dimensions rotate fast, high dimensions slowly.
 (b) The dot product is $\cos((n - m)\theta)$. Both pairs have $n - m = 3$: $\cos(1.5) \approx 0.0707$. Identical, which confirms the dependence only on relative position.

Problem 14 (ALiBi slopes and biases)

For $H = 8$ heads, list the slopes. What bias does a query add for a key 4 positions back in head 1 and in head 8?

Solution

$m_h = 2^{-8h/8} = 2^{-h} \cdot \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots, \frac{1}{256}$.
 Head 1: $-\frac{1}{2} \cdot 4 = -2$ (strong recency). Head 8: $-\frac{4}{256} \approx -0.0156$ (almost global). Different heads get different “attention spans”.

Problem 15 (Sinusoidal PE by hand)

Compute PE_p for $p = 3, d = 4$.

Solution

Frequencies $1/10000^{0/4} = 1$ and $1/10000^{2/4} = 0.01$.
 $PE_3 = (\sin 3, \cos 3, \sin 0.03, \cos 0.03) = (0.1411, -0.9900, 0.0300, 0.9996)$.

Problem 16 (LayerNorm vs. RMSNorm)

For $x = (1, 2, 3, 4)$, $\gamma = 1$, $\beta = 0$, $\epsilon = 0$, compute both.

Solution

LayerNorm: $\mu = 2.5$, $\sigma^2 = 1.25$, $\sigma = 1.118$: $(-1.342, -0.447, 0.447, 1.342)$ (zero mean, unit variance).
 RMSNorm: $\text{RMS} = \sqrt{30/4} = 2.739$: $(0.365, 0.730, 1.095, 1.461)$ (not centred; unit RMS).

Problem 17 (DeepNorm constants)

Compute DeepNorm’s α and β for a 96-layer decoder-only model.

Solution

$\alpha = (2N)^{1/4} = 192^{1/4} \approx 3.72$; $\beta = (8N)^{-1/4} = 768^{-1/4} \approx 0.19$. The residual is up-weighted $3.7\times$ and sub-layer weights are initialised $5\times$ smaller.

Problem 18 (Optimiser memory)

For a 70B model with fp32 optimiser states: memory for AdamW states vs. LION states?

Solution

AdamW: $2 \text{ states} \times 70 \times 10^9 \times 4 \text{ B} = 560\text{GB}$. LION: $1 \text{ state} = 280\text{GB}$. Both exclude fp32 master weights (280GB) and gradients.

Problem 19 (Sparse attention savings)

For $n = 2048$: count allowed (query, key) pairs for dense causal attention and for a local window of $w = 256$.

Solution

Dense causal: $n(n+1)/2 = 2,098,176$.

Local window: $nw - w(w-1)/2 = 524,288 - 32,640 = 491,648$ (the first w rows see fewer keys), about 23% of dense.

Problem 20 (Why FlashAttention saves memory)

$n = 8192$, fp16, 32 heads, batch 1: size of the materialised score matrices in standard attention? What does FlashAttention store instead?

Solution

Per head $n^2 \cdot 2 \text{ B} = 128\text{MiB}$; $\times 32 \text{ heads} = 4\text{GiB}$ for S alone (plus the same again for P if stored). FlashAttention stores only O and the per-row statistics m, ℓ : $O(n)$ memory per head.

Problem 21 (PagedAttention fragmentation)

Block size 16 tokens; a request currently holds 1000 tokens. (a) How many blocks and how many wasted slots? (b) Compare with pre-allocating a contiguous 2048-token buffer.

Solution

(a) $\lceil 1000/16 \rceil = 63 \text{ blocks} = 1008 \text{ slots}$, so 8 are wasted (0.8%).

(b) Contiguous: $2048 - 1000 = 1048 \text{ unused slots}$ (51%). Paging wastes at most one partial block per sequence.

Problem 22 (GPT-1 parameters)

Verify 117M: $L = 12$, $d_{\text{model}} = 768$, $V = 40478$, $T = 512$.

Solution

$12 \cdot 12 \cdot 768^2 = 84,934,656$; $40478 \cdot 768 = 31,087,104$; $512 \cdot 768 = 393,216$. Total = $116,414,976 \approx 117\text{M}$ ✓.

Part C

Subjective questions with model answers

Q1. Describe the three branches of the LLM evolutionary tree and the trajectory of each.

Solution

Encoder-only (BERT, RoBERTa, ALBERT, ELECTRA, DeBERTa): bidirectional MLM-style models, dominant for a short time after 2018 for understanding tasks, then largely stopped growing. **Encoder-decoder** (T5, BART, mT5, T0, Switch, UL2, Flan-T5): championed by T5's text-to-text framework; a steady but smaller branch. **Decoder-only** (GPT-1/2/3, GPT-J, Gopher, PaLM, OPT, BLOOM, LLaMA, ChatGPT, GPT-4, Claude): championed by GPT-2's zero-shot prompting; after scaling laws and GPT-3 it became by far the largest branch and dominates today.

Q2. What questions did scaling laws raise, and how did the answer change between Kaplan (2020) and Chinchilla (2022)?

Solution

Given more compute, should we scale the model, the data, or both, and in what proportion? Kaplan et al. found loss follows power laws in N , D and C and recommended putting most extra compute into parameters ($N \propto C^{0.73}$, $D \propto C^{0.27}$). This sparked the race for ever-larger models (GPT-3, Jurassic-1, Gopher, MT-NLG). Chinchilla re-fitted with learning-rate schedules matched to each training length and found N and D should scale roughly equally ($\approx C^{0.5}$ each), about 20 tokens per parameter. Chinchilla (70B, 1.4T tokens) beat Gopher (280B, 300B tokens) at the same compute, showing that earlier giants were under-trained.

Q3. Why did the field converge on decoder-only architectures?

Solution

(1) A single causal LM can perform any task framed as text continuation (prompting, in-context learning), so no task-specific heads or separate encoders are needed. (2) CLM gives a training signal at every token and matches generation exactly. (3) One homogeneous stack is easy to scale and serve (KV caching). (4) Emergent few-shot abilities at GPT-3 scale and the success of chat assistants (InstructGPT $\rightarrow$ ChatGPT) rewarded open-ended generation. (5) Open releases such as OPT and LLaMA gave the community strong decoder-only bases to build on.

Q4. Explain RLHF as used in InstructGPT and why it was needed.**Solution**

A pre-trained LM imitates web text rather than following instructions helpfully or safely. InstructGPT: (1) **SFT** on human-written demonstrations; (2) a **reward model** trained on human rankings of sampled outputs with a pairwise loss $-\log \sigma(r_w - r_l)$; (3) **RL (PPO)** to maximise reward minus a KL penalty to the SFT model, which prevents reward hacking and preserves language quality. The result followed instructions much better. Labellers preferred a 1.3B InstructGPT to the 175B GPT-3. ChatGPT was built on these ideas.

Q5. “All decoder-only LLMs are the same architecture.” In what ways do they still differ?**Solution**

They share the macro-structure (stacked masked self-attention + FFN) but differ in: the **data** (sources, mixture, filtering; mostly Common Crawl-derived); the **objective** (usually CLM); the **optimiser** (AdamW, Adafactor, LION); **normalisation** type (LayerNorm, RMSNorm, DeepNorm) and placement (post, pre, sandwich); **activation**/FFN (ReLU, GELU, SwiGLU, GeGLU; dense or MoE); the **attention** variant (dense or sparse, MHA/MQA/GQA, Flash or Paged kernels); the **position encoding** (absolute, relative, RoPE, NoPE, ALiBi); and depth/width and training steps. These choices mainly aim to reduce compute or memory and improve quality or stability.

Q6. Compare Post-Norm and Pre-Norm. Why did large models move to Pre-Norm, and what do DeepNorm and Sandwich-Norm try to fix?**Solution**

Post-Norm, $\text{LN}(x + F(x))$, normalises after the residual addition, so there is no clean identity path. Gradients near the output are large, deep models diverge without careful warm-up. Pre-Norm, $x + F(\text{LN}(x))$, keeps an un-normalised residual stream with an identity path, $\partial x_L / \partial x_0 = I + \dots$, so training is stable at depth (GPT-2/3, PaLM, LLaMA). Its downside is that later layers contribute relatively small updates, so it can under-use depth and let activations grow. DeepNorm keeps Post-Norm’s quality but up-weights the residual (αx) and down-scales the initialisation to bound update size, enabling 1000-layer models. Sandwich-Norm adds a second LN on the sub-layer output to control activation magnitudes.

Q7. Why is RMSNorm preferred in many recent models?**Solution**

RMSNorm drops LayerNorm’s mean-centring and bias, normalising only by the root-mean-square: $x / \text{RMS}(x) \cdot \gamma$. Empirically the re-scaling property gives most of LayerNorm’s benefit, so RMSNorm matches quality while needing fewer reductions and parameters, which means faster kernels at scale (Gopher, LLaMA, Mistral).

Q8. Explain SwiGLU and why its hidden dimension is typically $\frac{8}{3}d_{\text{model}}$.**Solution**

SwiGLU replaces $f(xW_1)W_2$ with $(\text{Swish}(xW) \otimes xV)W_2$: a smooth non-linear gate multiplies a linear value path element-wise, which gives multiplicative interactions and lower perplexity at equal parameters (Shazeer 2020). It has three weight matrices instead of two, so to keep the FFN at $8d_{\text{model}}^2$ parameters we need $3d_{\text{model}}d_{\text{ff}} = 8d_{\text{model}}^2$, giving $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ (e.g. 11008 for LLaMA-7B after rounding to a multiple of 256).

Q9. What is a Mixture of Experts? Discuss benefits and costs using GLaM.**Solution**

MoE replaces the FFN with many expert FFNs and a router that sends each token to its top- k experts; the output is the gate-weighted sum. Total parameters grow with the number of experts while per-token compute grows only with k . GLaM has 1.2T parameters but $\approx 97\text{B}$ active per token (64 experts, top-2), so its inference FLOPs were below GPT-3's with better quality. Costs: all experts must be stored (memory), tokens must be routed across devices (communication), training can be unstable, and an auxiliary load-balancing loss is needed so experts are used evenly. Comparing a sparse MoE's parameter count with a dense model's is therefore misleading.

Q10. Compare MHA, MQA and GQA. Why do they matter mostly at inference?**Solution**

MHA gives each query head its own K and V; MQA shares one K/V head across all query heads; GQA shares one K/V head per group of query heads. During autoregressive decoding the KV cache ($2Lh_{kv}d_{\text{head}}$ values per token) must be read at every step, so decoding is memory-bandwidth bound. Reducing h_{kv} shrinks the cache and the bytes read per step ($8\times$ for GQA-8 on LLaMA-2-70B, $64\times$ for MQA), which enables longer contexts and bigger batches. MQA can lose some quality; GQA recovers almost all of MHA's quality with most of MQA's speed.

Q11. FlashAttention and PagedAttention both concern "memory". How do they differ?**Solution**

FlashAttention is a *compute kernel*: it computes exact attention by tiling Q, K, V through on-chip SRAM with an online softmax, never materialising the $n \times n$ matrix in HBM. This cuts memory from $O(n^2)$ to $O(n)$ and reduces slow memory traffic, so training and prefill are faster. PagedAttention is a *memory-management* scheme for serving: it stores each sequence's KV cache in fixed-size blocks mapped by a block table (like OS paging). This removes fragmentation and over-reservation and allows sharing blocks across sequences, so more requests fit per GPU. One speeds up the attention computation; the other packs the KV cache efficiently.

Q12. Derive why RoPE encodes relative position, and contrast it with absolute and ALiBi encodings.

Solution

RoPE rotates each 2-D pair of q and k by angle $m\theta_i$ at position m . Since $R(a)^\top R(b) = R(b - a)$, $(R_m q)^\top (R_n k) = q^\top R_{n-m} k$, so logits depend only on the offset $n - m$. Absolute encodings add a position vector to the input; the model sees absolute indices and cannot extrapolate beyond the trained length. ALiBi adds no vectors but penalises logits linearly with distance using head-specific fixed slopes, which gives a recency bias and good length extrapolation. RoPE sits between them: relative behaviour and no added parameters, and it can be extended to longer contexts by rescaling its frequencies.

Q13. How can a model with no position encoding (NoPE) know word order?

Solution

The causal mask itself breaks permutation symmetry: position i attends to exactly i tokens. The model can, for example, attend uniformly to all previous tokens and read off quantities that scale with $1/i$, inferring its absolute position. It can then build relative-position computations internally. This only works with a causal (not fully visible) mask, which is why NoPE applies to decoder-only models.

Q14. Walk through the GPT-1 $\rightarrow$ GPT-3 $\rightarrow$ PaLM case studies. What changed and what stayed?

Solution

Stayed the same: decoder-only architecture, CLM objective, next-token generation. GPT-1: 12 layers, post-LN, GELU, MHA, learned absolute positions, BookCorpus, Adam(W). GPT-3: 96 layers, pre-LN, GELU, alternating dense and *sparse* attention, absolute positions, CommonCrawl/WebText2/Books/Wikipedia (300B tokens), AdamW with $\beta_2 = 0.95$. PaLM: 118 layers, pre-LN (parallel attention+FFN), *SwiGLU*, multi-query attention (the slide marks MHA), *RoPE*, *Adafactor*, and a 780B-token mix of conversations, web, books, code, Wikipedia and news. Trend: scale up; move to stability-friendly normalisation; better activations; relative-style positions; attention variants that make inference cheaper.

Q15. What is the “road ahead”, and why does it start with data?

Solution

The course will walk the unifying diagram from the bottom: data sources $\rightarrow$ data pipelines $\rightarrow$ filtering rules $\rightarrow$ datasets (C4, mC4, The Pile, RedPajama, Dolma, ...), then position encodings, attention variants, normalisation, activations/MoE. Data comes first because it largely determines what the model can learn, and Week 7 showed that quality and deduplication matter (unfiltered C4 hurt; heavy repetition caused memorisation). Most modern corpora are Common Crawl plus a sequence of filtering rules: language ID, quality heuristics or classifiers, deduplication, and safety filtering.

Q16. Discuss the open vs. closed divide in LLMs. Why did OPT and LLaMA matter?**Solution**

Closed models (GPT-3, PaLM, ChatGPT, GPT-4, Claude) are available only through APIs, often with undisclosed architecture and data, which limits research and reproducibility. OPT-175B gave researchers a GPT-3-scale model with a public logbook; LLaMA/LLaMA-2 released strong, compute-efficient models (trained well past 20 tokens/parameter) that the community fine-tuned, quantised and extended, enabling instruction-tuned open models and much architecture research. Open data efforts (The Pile, RedPajama, Dolma) similarly make the data side studyable.

Part D

Multiple-choice questions with answers and explanations

Questions marked [MSQ] may have more than one correct option.

D.1 The big picture

Q1. In the lecture, the first way of grouping LLMs is by:

- (a) number of parameters
- (b) type of architecture (encoder, encoder–decoder, decoder)
- (c) training dataset
- (d) release year

Answer: (b). The evolutionary tree has three branches by architecture.

Q2. Which branch dominated the field for a short time right after 2018?

- (a) Encoder–decoder
- (b) Decoder-only
- (c) Encoder-only (BERT and variants)
- (d) Mixture of experts

Answer: (c). BERT and its variants, before GPT-2/T5 shifted attention.

Q3. GPT-2 and T5 respectively advocated for:

- (a) encoder-only and decoder-only
- (b) both encoder-only
- (c) encoder–decoder and decoder-only
- (d) decoder-only and encoder–decoder

Answer: (d). GPT-2: decoder-only prompting; T5: text-to-text encoder–decoder.

Q4. Which of the following was *open-sourced*?

- (a) GPT-J 6B
- (b) Jurassic-1 178B
- (c) GPT-3 175B
- (d) Gopher 280B

Answer: (a). GPT-J (EleutherAI) and OPT-175B (Meta) were open; the others are proprietary.

Q5. GLaM has 1.2T parameters. Its per-token compute is closest to a dense model of about:

- (a) 600B
- (b) 10B
- (c) 1.2T
- (d) 100B

Answer: (d). Mixture of experts with top-2 of 64 experts: $\approx 97\text{B}$ active parameters per token.

Q6. InstructGPT introduced which alignment approach for conversational models?

- (a) Contrastive pre-training
- (b) RLHF
- (c) Knowledge distillation
- (d) Span corruption

Answer: (b). SFT, then a reward model from human rankings, then PPO; ChatGPT builds on this.

Q7. Most pre-training datasets are derived primarily from:

- (a) BookCorpus
- (b) Wikipedia
- (c) Common Crawl
- (d) GitHub

Answer: (c). The slide: “common crawl being the source for majority of the datasets”.

D.2 Scaling

Q8. Under Chinchilla’s compute-optimal rule, a model with N parameters should be trained on about:

- (a) $2N$ tokens
- (b) $1000N$ tokens
- (c) $20N$ tokens
- (d) $N/20$ tokens

Answer: (c). About 20 tokens per parameter.

Q9. Training compute for a dense transformer is approximately:

- (a) $6ND$
- (b) N^2D
- (c) ND^2
- (d) $2ND$

Answer: (a). $2N$ per token forward plus $4N$ backward.

Q10. Kaplan et al. recommended that extra compute go mostly into:

- (a) more epochs on the same data
- (b) more data
- (c) longer context
- (d) model size

Answer: (d). $N_{\text{opt}} \propto C^{0.73}$ vs. $D_{\text{opt}} \propto C^{0.27}$, hence the “race to build larger models”.

Q11. GPT-3 trained on 300B tokens has roughly how many tokens per parameter?

- (a) 0.17
- (b) 17
- (c) 1.7
- (d) 170

Answer: (c). $300/175 \approx 1.7$: under-trained by Chinchilla's criterion.

D.3 Design axes

Q12. RMSNorm differs from LayerNorm by:

- (a) using the median
- (b) normalising across the batch
- (c) adding a learned position term
- (d) omitting mean-centring (and bias)

Answer: (d). $x / \text{RMS}(x) \cdot \gamma$: re-scaling only.

Q13. Pre-Norm computes:

- (a) $\text{LN}(\alpha x + F(x))$
- (b) $\text{LN}(x + F(x))$
- (c) $x + F(\text{LN}(x))$
- (d) $x + \text{LN}(F(\text{LN}(x)))$

Answer: (c). (b) Post-Norm, (d) Sandwich, (a) DeepNorm.

Q14. The main reason large models prefer Pre-Norm is:

- (a) better extrapolation to long contexts
- (b) it removes the need for attention
- (c) an identity path through the residual stream that stabilises deep training
- (d) fewer parameters

Answer: (c). Gradients reach early layers without passing through repeated LNs.

Q15. In DeepNorm for an N -layer decoder, the residual scaling α is:

- (a) $(8N)^{-1/4}$
- (b) $(2N)^{1/4}$
- (c) $\sqrt{N}$
- (d) $1/N$

Answer: (b). The initialisation scale is $\beta = (8N)^{-1/4}$.

Q16. SwiGLU uses how many weight matrices in the FFN?

- (a) 4
- (b) 1
- (c) 2
- (d) 3

Answer: (d). W (gate), V (value), W_2 (output), hence $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ for equal parameters.

Q17. Swish is defined as:

- (a) $x\Phi(x)$

- (b) $\tanh(x)$
- (c) $\max(0, x)$
- (d) $x\sigma(\beta x)$

Answer: (d). With $\beta = 1$ it is SiLU; GELU is $x\Phi(x)$.

Q18. Which activation did PaLM use?

- (a) SwiGLU
- (b) GeGLU
- (c) GeLU
- (d) ReLU

Answer: (a). PaLM: SwiGLU. GPT-1 and GPT-3: GeLU.

Q19. In multi-query attention (MQA):

- (a) attention is computed only locally
- (b) each head has its own K and V
- (c) all query heads share a single K and V head
- (d) queries are shared across heads

Answer: (c). GQA shares K/V within groups; MHA does not share.

Q20. GQA with g groups and h heads reduces the KV cache relative to MHA by a factor of:

- (a) h/g
- (b) $\sqrt{h}$
- (c) $h \cdot g$
- (d) g

Answer: (a). KV heads go from h to g .

Q21. FlashAttention is best described as:

- (a) a new position encoding
- (b) a KV-cache paging scheme
- (c) an approximate sparse attention
- (d) an exact, IO-aware attention kernel using tiling and online softmax

Answer: (d). Exact results; avoids materialising $n \times n$ in HBM.

Q22. PagedAttention primarily addresses:

- (a) training instability
- (b) vanishing gradients
- (c) KV-cache memory fragmentation during serving
- (d) tokenisation errors

Answer: (c). Block tables map logical KV blocks to physical blocks, like virtual memory.

Q23. GPT-3 is marked with which attention type in the lecture's diagram?

- (a) Sparse
- (b) GQA
- (c) Paged
- (d) Flash

Answer: (a). GPT-3 alternates dense and locally banded sparse attention layers.

Q24. RoPE makes the attention score depend on:

- (a) nothing positional
- (b) absolute position of the query only
- (c) the relative offset $n - m$
- (d) the sum $m + n$

Answer: (c). $(R_m q)^\top (R_n k) = q^\top R_{n-m} k$.

Q25. ALiBi slopes for 8 heads are:

- (a) learned
- (b) $1, 2, \dots, 8$
- (c) $\frac{1}{2}, \frac{1}{4}, \dots, \frac{1}{256}$
- (d) all equal to 1

Answer: (c). Geometric sequence $2^{-8h/H}$, fixed and not learned.

Q26. Which position encoding is known for good length extrapolation via a linear distance penalty?

- (a) Sinusoidal absolute
- (b) ALiBi
- (c) Learned absolute
- (d) None of these

Answer: (b). Used in BLOOM and MPT.

Q27. [MSQ] Which are position-encoding options on the unifying diagram?

- (a) ALiBi
- (b) RoPE
- (c) GQA
- (d) NoPE

Answer: (a), (b), (d). GQA is an attention variant.

Q28. LION differs from AdamW mainly in that it:

- (a) needs no learning rate
- (b) uses the sign of an interpolated momentum and keeps one state
- (c) is second-order
- (d) stores two states per parameter

Answer: (b). Half the optimiser memory; uniform-magnitude updates.

Q29. AdamW's "decoupled" weight decay means:

- (a) decay is applied directly to weights, outside the adaptive scaling
- (b) no weight decay
- (c) decay is added to the gradient before adaptive scaling
- (d) decay only on biases

Answer: (a). Every weight shrinks at the same relative rate.

D.4 Case studies

Q30. GPT-1 uses:

- (a) Post-Norm LayerNorm, GeLU, absolute positions, $N = 12$
- (b) Pre-Norm, RoPE, SwiGLU
- (c) RMSNorm, ALiBi
- (d) DeepNorm, MQA

Answer: (a). Trained on BookCorpus with CLM.

Q31. How many layers does GPT-3 175B have?

- (a) 48
- (b) 118
- (c) 128
- (d) 96

Answer: (d). $d_{\text{model}} = 12288$, 96 heads.

Q32. PaLM 540B used which optimiser and position encoding?

- (a) Adam and relative
- (b) LION and ALiBi
- (c) Adafactor and RoPE
- (d) AdamW and absolute

Answer: (c). Also SwiGLU and $N = 118$.

Q33. [MSQ] Which datasets are highlighted for GPT-3 in the lecture?

- (a) Wikipedia
- (b) StarCoder
- (c) CommonCrawl
- (d) WebText

Answer: (a), (c), (d). Plus books. StarCoder* (a look-alike for code) appears for PaLM.

Q34. In the PaLM diagram, asterisks on CC-News*, RealNews*, StarCoder* indicate:

- (a) datasets similar to those used (PaLM's mix is proprietary)
- (b) datasets removed after filtering
- (c) datasets that were deduplicated
- (d) multilingual datasets

Answer: (a). PaLM's actual mix (conversations, web, books, GitHub, Wikipedia, news) is not public.

Q35. The “road ahead” in the lecture starts with:

- (a) RLHF
- (b) position encodings
- (c) MoE
- (d) data sources, data pipelines and datasets

Answer: (d). Common Crawl → filtering rules → datasets, then moving up the diagram.

D.5 Answer key

1 b	2 c	3 d	4 a	5 d	6 b	7 c	8 c	9 a	10 d
11 c	12 d	13 c	14 c	15 b	16 d	17 d	18 a	19 c	20 a
21 d	22 c	23 a	24 c	25 c	26 b	27 abd	28 b	29 a	30 a
31 d	32 c	33 acd	34 a	35 d					
